



<https://haoailab.com/cse291-s26/>

CSE/DSC 291: Deep Learning Systems Spring 2026

LLM, diffusion, and case studies

Optimizations and Parallelization

Basics

MCQ Time

What is a “kernel” in the context of GPUs?

- A. A specific section of the CPU used for memory operations.
- B. A specific section of the GPU used for memory operations.
- C. A type of thread that operates on the GPU.
- D. A function that is executed simultaneously by tens of thousands of threads on GPU cores.

What is the function of shared memory in the context of GPU execution?

- A. It's HBM
- B. It's used to store all the threads in a block.
- C. It can be used to “cache” data that is used by more than one thread, avoiding multiple reads from the global memory.
- D. It's used to store all the CUDA cores.

What is the significance of over-subscribing the GPU?

- A. It reduces the overall performance of the GPU.
- B. It ensures that there are more blocks than SMPs present on the device, helping to hide latencies and ensure high occupancy of the GPU.
- C. It leads to a memory overflow in the GPU.
- D. It ensures that there are more SMPs than blocks present on the device.

Which of the following is True about GPU Memory

- A. On H100, a CPU process can access an array stored on H100 GPU memory
- B. A thread in a threadblock can access its threadblock-level shared memory
- C. Pinned memory is a part of memory allocated on GPU
- D. `print(a)` function in C++ can print an array allocated via `a = cudaMalloc(..)`

Which of the following operations is most likely to be limited by arithmetic operations?

- A. ReLU Activation
- B. Linear layer (8192 outputs, 2048 inputs, batch size 1)
- C. Batch normalization
- D. Max pooling (3x3 window and unit stride)
- E. Layer normalization
- F. Linear layer (2048 outputs, 1024 inputs, batch size 512)

When picking a tile size for GEMM, why not always pick the biggest tile size?

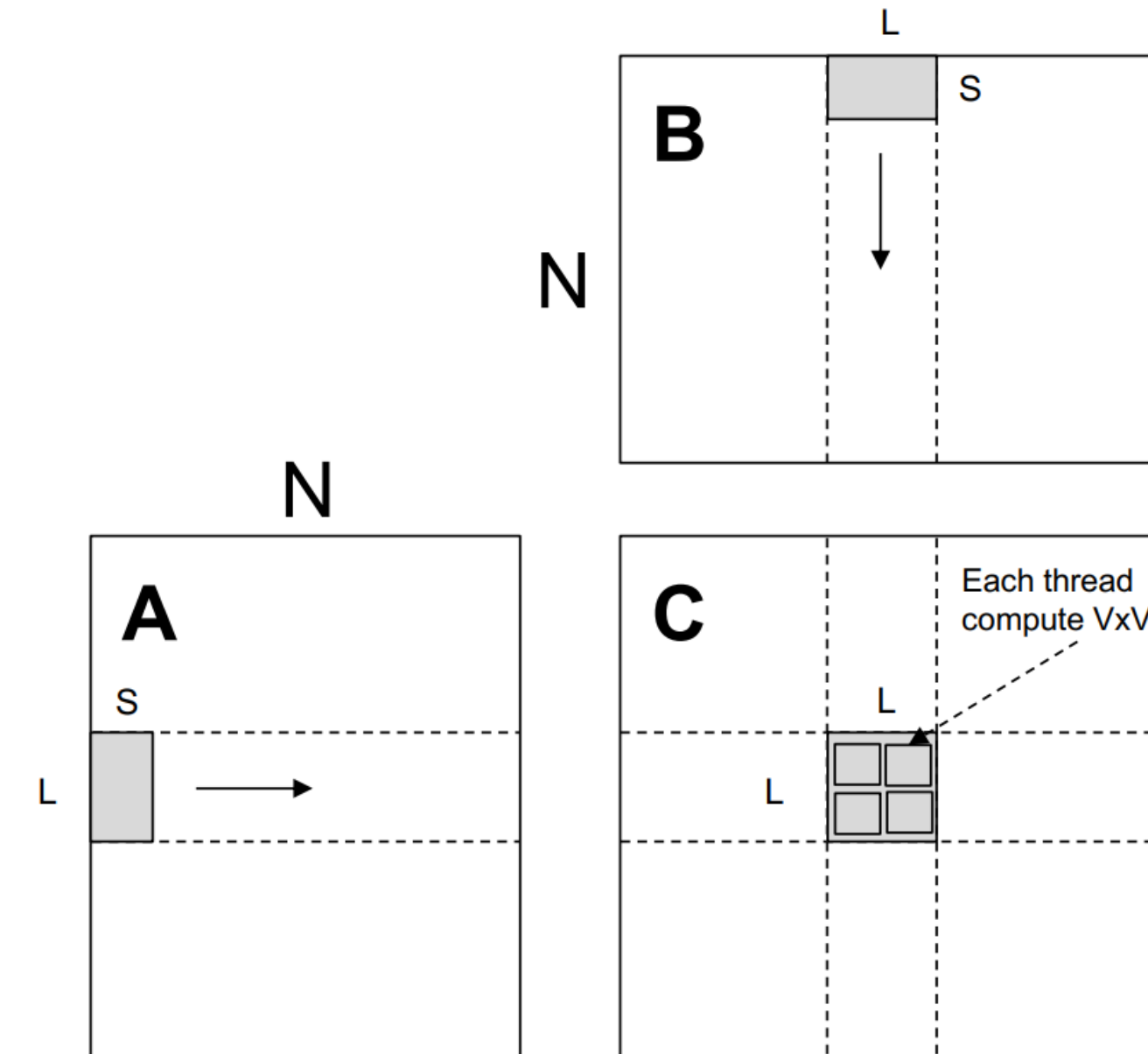
- A. The tile might not fit on the GPU HBM for some GEMM sizes
- B. The bigger size could result in low parallelism for some GEMM sizes
- C. Larger tiles have lower data reuse
- D. Larger tiles means more data is read, lowering arithmetic intensity.

Recap of Matmul Problem

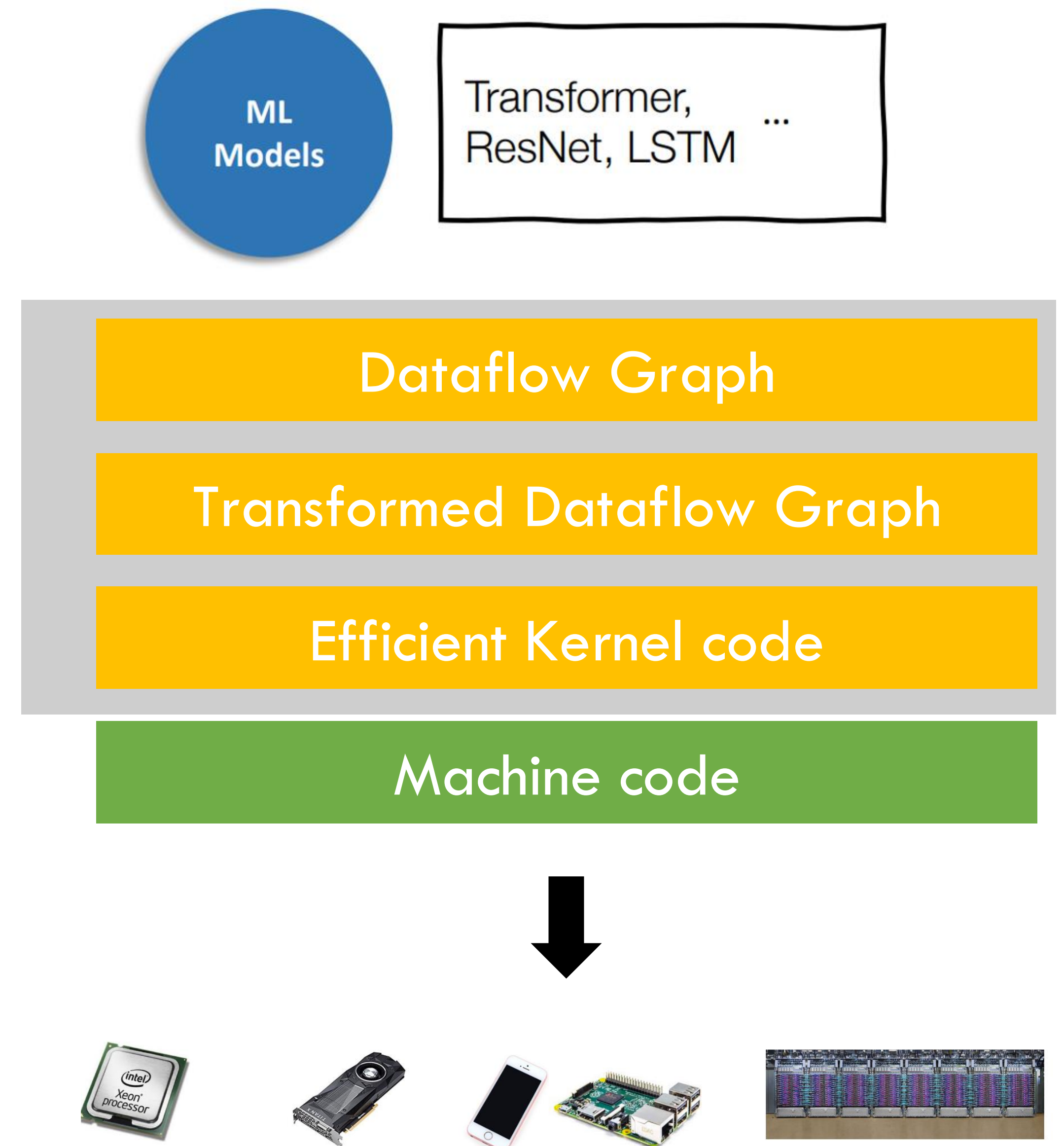
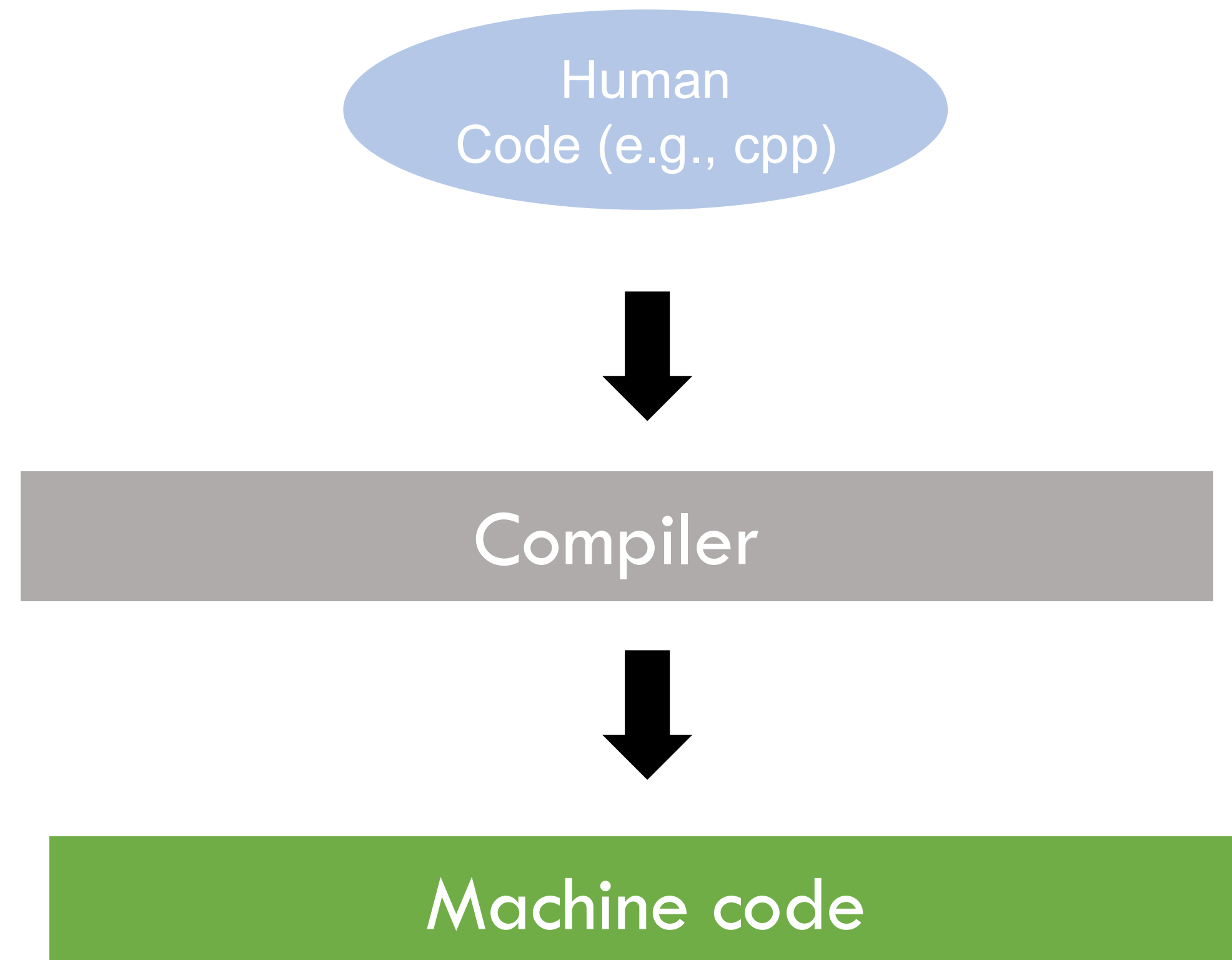
- How to implement an highly efficient kernel
- How to choose configs.

- #threads
- #registers
- Amount of SRAM

- Solution 1:
 - expert-craft -> Enumerate configs -> profile
- Solution 2: Operator compilation



Traditional vs. ML Program



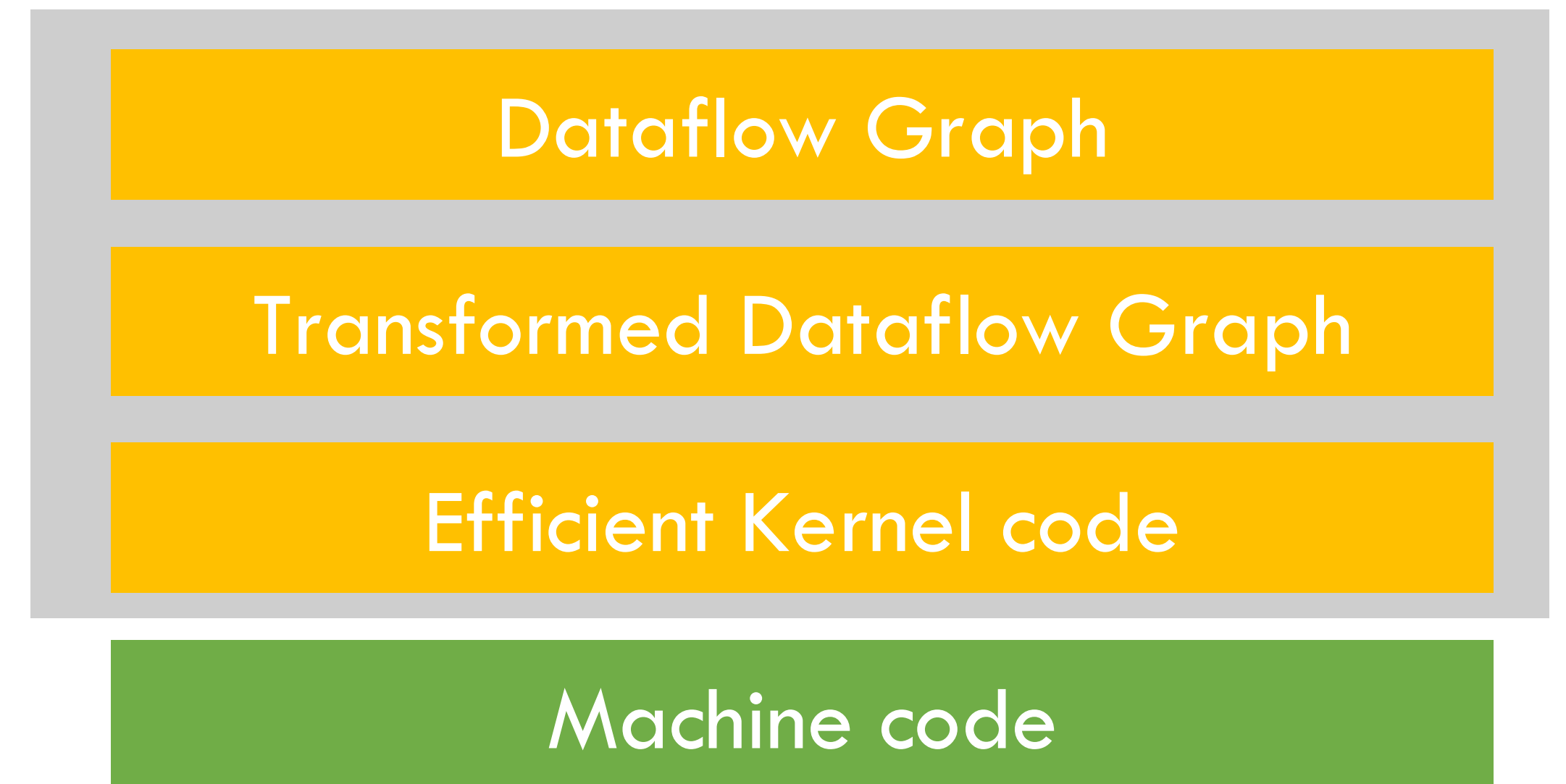
ML Compilation: Big Picture

ML compilation's Goal:

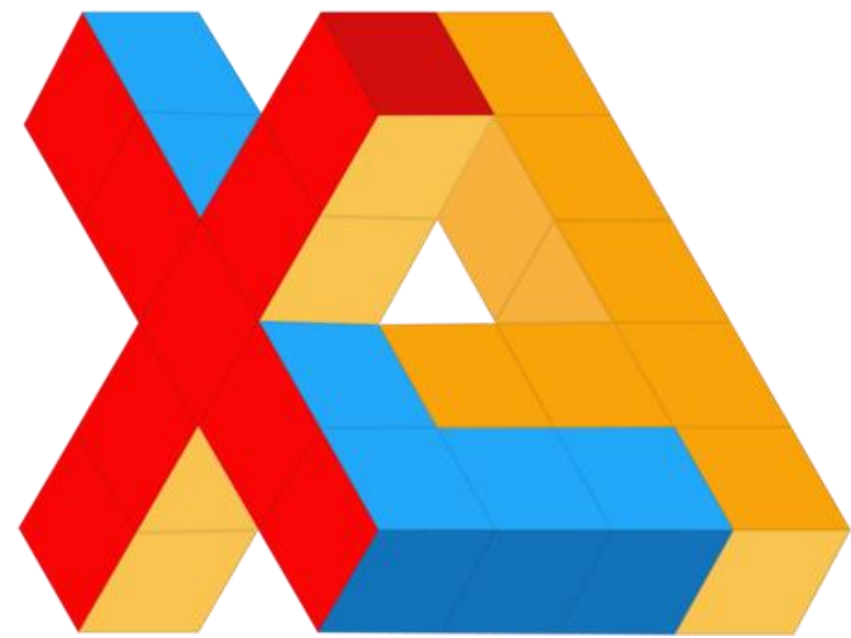
Automatically generate *optimal* configurations and code
given users code and target hardware

Grand Problems to Build ML Compilers

- **Programming-level: program translation**
 - Automatically transform an arbitrary (usually imperative) code (by developers) into a compile-able code (e.g., static dataflow graph)?
- **Graph-level: graph compilation**
 - Automatic graph transformations to make it faster
- **Op-level: operation compilations**
 - How to make operator fast on different hardware?

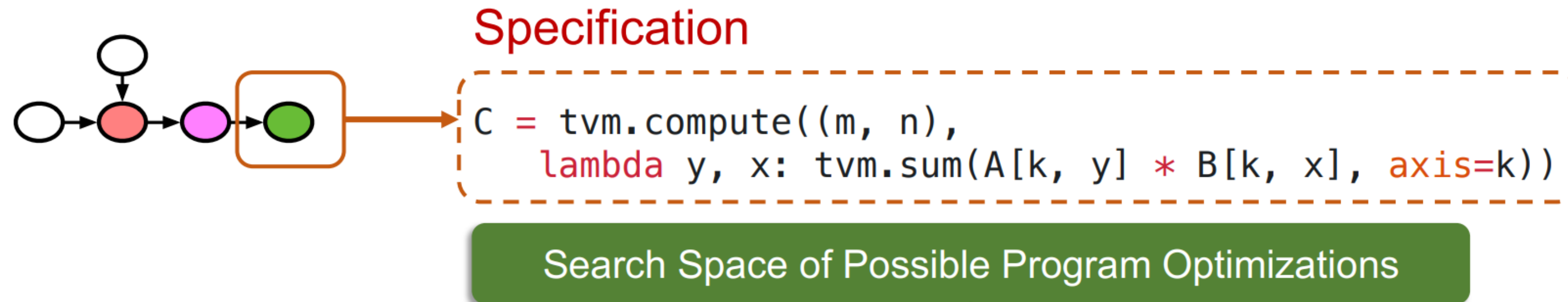


Notable Compilers



Modular

Example: Operator Compilation



Low-level Program Variants

```
inp_buffer AL[8][8], BL[8][8]
acc_buffer CL[8][8]
for yo in range(128):
    for xo in range(128):
        vdlc.fill_zero(CL)
        for ko in range(128):
            vdlc.dma_copy2d(AL, A[ko*8:ko*8+8][yo*8:yo*8+8])
            vdlc.dma_copy2d(BL, B[ko*8:ko*8+8][xo*8:xo*8+8])
            vdlc.fused_gemm8x8_add(CL, AL, BL)
            vdlc.dma_copy2d(C[yo*8:yo*8+8,xo*8:xo*8+8], CL)
```

```
for yo in range(128):
    for xo in range(128):
        C[yo*8:yo*8+8][xo*8:xo*8+8] = 0
        for ko in range(128):
            for yi in range(8):
                for xi in range(8):
                    for ki in range(8):
                        C[yo*8+yi][xo*8+xi] +=
                            A[ko*8+ki][yo*8+yi] * B[ko*8+ki][xo*8+xi]
```

```
for y in range(1024):
    for x in range(1024):
        C[y][x] = 0
        for k in range(1024):
            C[y][x] += A[k][y] * B[k][x]
```

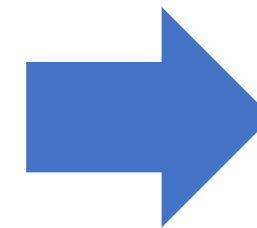

Transforming Loops: Loop Splitting

Code

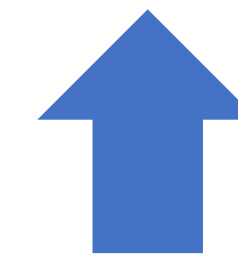
```
for x in range(128):  
    C[x] = A[x] + B[x]
```



```
for xo in range(32):  
    for xi in range(4):  
        C[xo * 4 + xi]  
        = A[xo * 4 + xi] + B[xo * 4 + xi]
```



```
def gpu_kernel():  
    C[threadIdx.x * 4 + blockIdx.x] = . . .
```

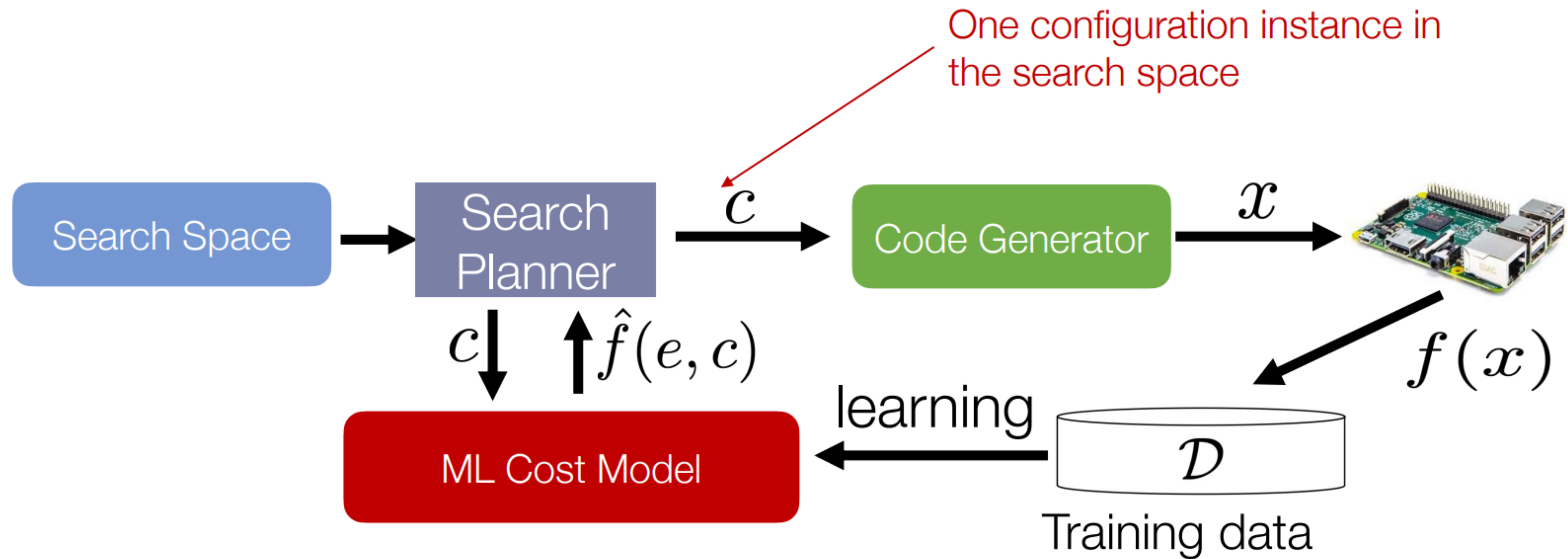


```
for xi in range(4):  
    for xo in range(32):  
        C[xo * 4 + xi]  
        = A[xo * 4 + xi] + B[xo * 4 + xi]
```

Problems

- We need to enumerate many possibilities
 - How to represent all “possibilities”
- We need to find the (close-to-)optimal values (register/cache sizes)
 - How to search?
- We need to apply this to so many operators and devices
 - How to reduce search space
 - How to generalize?

Search via Learned Cost Model (or LLM)



Discussion

ML compilation's Promise:

Automatically generate *optimal* configurations and code
given users ML code on target hardware

Q: How well are we achieving our goals in ML compilers?

Next

- Operator Compilation
- **High-level DSL for CUDA: Triton**
- Graph Optimization

Dataflow Graph

Autodiff

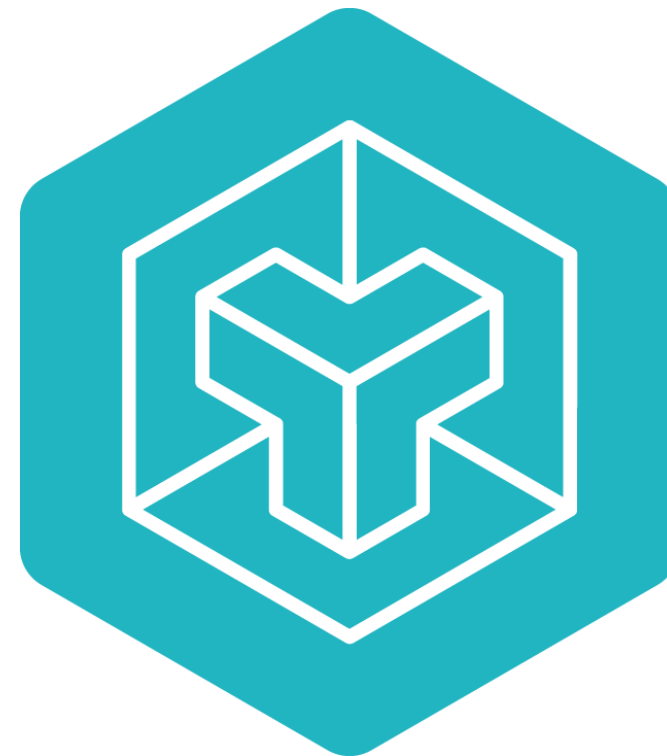
Graph Optimization

Parallelization

Runtime: schedule /
memory

Operator

Which Triton?



Device-specific DSL (e.g., CUDA) vs. Compiler

+ developers can do whatever the heck they want:

- squeeze the last bits of performance
- use whatever data-structure you want

-- developers can do whatever the heck they want:

- Require deep expertise; performance optimization is very time-consuming
- Codebases are complex and hard to maintain

+ Very fast iteration speed for developers

- Can prototype ideas quickly and give it to compiler

-- Cannot represent certain types of ideas

- In-operator control flow
- Custom data structure

-- Code generation is a old difficult problem

- heavy use of templates and pattern-matching
- lots of performance cliffs

Device-specific DSL

Compiler

Triton's Pitch (Please think about why this makes sense)

+ simpler than CUDA;
more expressive than
graph compilers:



-- less expressive than
CUDA; more complicated
than graph compilers;



Triton Programming Model

- Users define **tensors** in **SARM**, and modify them using **torch-like primitives**

Embedded in Python



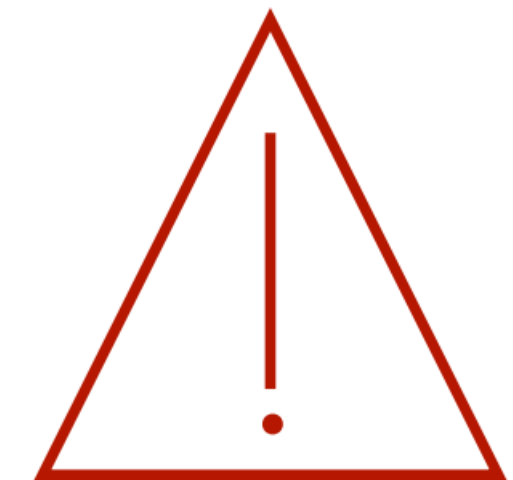
Kernels are defined in Python using `triton.jit`

Pointer arithmetics



Users construct tensors of pointers and (de)reference them elementwise

Shape Constraints



Must have power-of-two number of elements along each dimension

Example: elementwise add v1 ($z = x + y$)

- Triton kernel will be mapped to a single block (SM) of threads
- Users will be responsible for mapping to multiple blocks

```
import triton.language as tl
Import triton

@triton.jit
def _add(z_ptr, x_ptr, y_ptr, N):
    # same as torch.arange
    offsets = tl.arange(0, 1024)
    # create 1024 pointers to X, Y, Z
    x_ptrs = x_ptr + offsets
    y_ptrs = y_ptr + offsets
    z_ptrs = z_ptr + offsets
    # load 1024 elements of X, Y, Z
    x = tl.load(x_ptrs)
    y = tl.load(y_ptrs)
    # do computations
    z = x + y
    # write-back 1024 elements of X, Y, Z
    tl.store(z_ptrs, z)

N = 1024
x = torch.randn(N, device='cuda')
y = torch.randn(N, device='cuda')
z = torch.randn(N, device='cuda')
grid = (1, )
_add[grid](z, x, y, N)
```


Example: elementwise add v2 ($z = x + y$)

Use multiple blocks

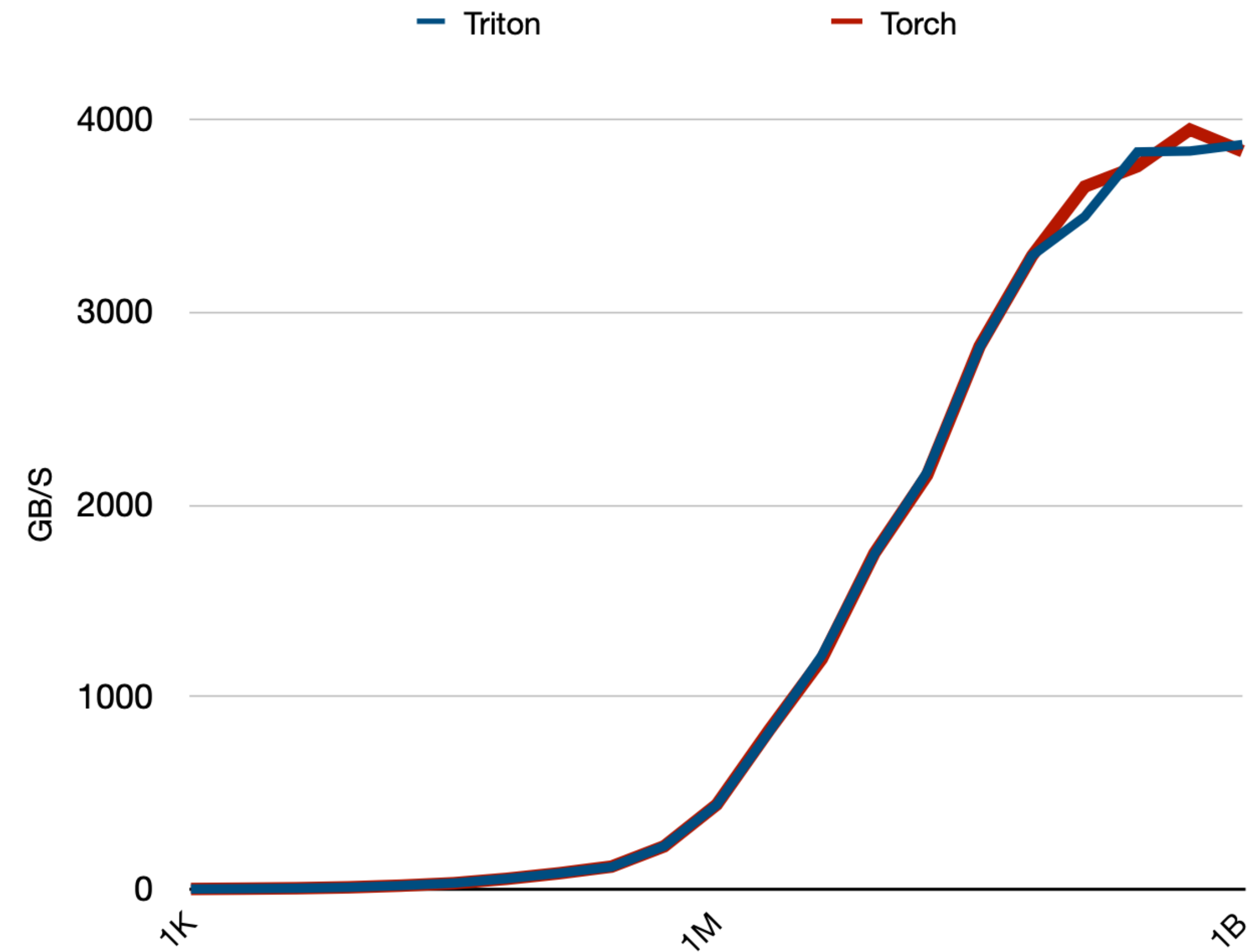
- Index the block and apply offset
- Adds bound check

```
import triton.language as tl
Import triton

@triton.jit
def _add(z_ptr, x_ptr, y_ptr, N):
    # same as torch.arange
    offsets = tl.arange(0, 1024)
    offsets += tl.program_id(0)*1024
    # create 1024 pointers to X, Y, Z
    x_ptrs = x_ptr + offsets
    y_ptrs = y_ptr + offsets
    z_ptrs = z_ptr + offsets
    # load 1024 elements of X, Y, Z
    x = tl.load(x_ptrs, mask=offset<N)
    y = tl.load(y_ptrs, mask=offset<N)
    # do computations
    z = x + y
    # write-back 1024 elements of X, Y, Z
    tl.store(z_ptrs, z)

N = 192311
x = torch.randn(N, device='cuda')
y = torch.randn(N, device='cuda')
z = torch.randn(N, device='cuda')
grid = (triton.cdiv(N, 1024), )
_add[grid](z, x, y, N)
```

Elementwise Add Performance

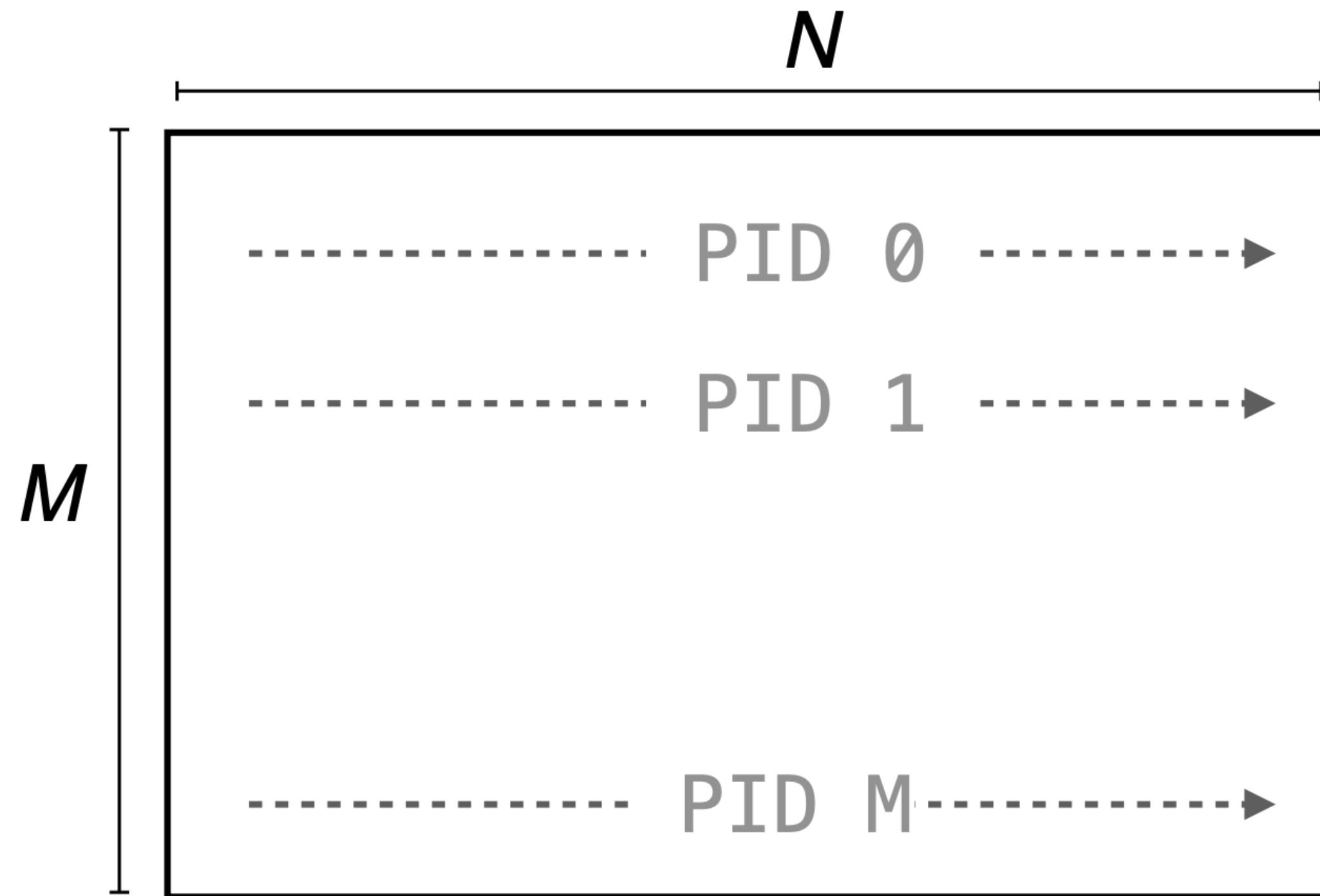


Another Example: Softmax

$$y_i = \text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum e^{x_d}}$$

- How did you implement this in PA1?
 - Think about the potential overhead when compose softmax from primitives
- Performant option: implementing an end-to-end softmax kernel
 - Think about the complexity of implementing in CUDA

Triton Example: softmax



```
import triton.language as tl
Import triton

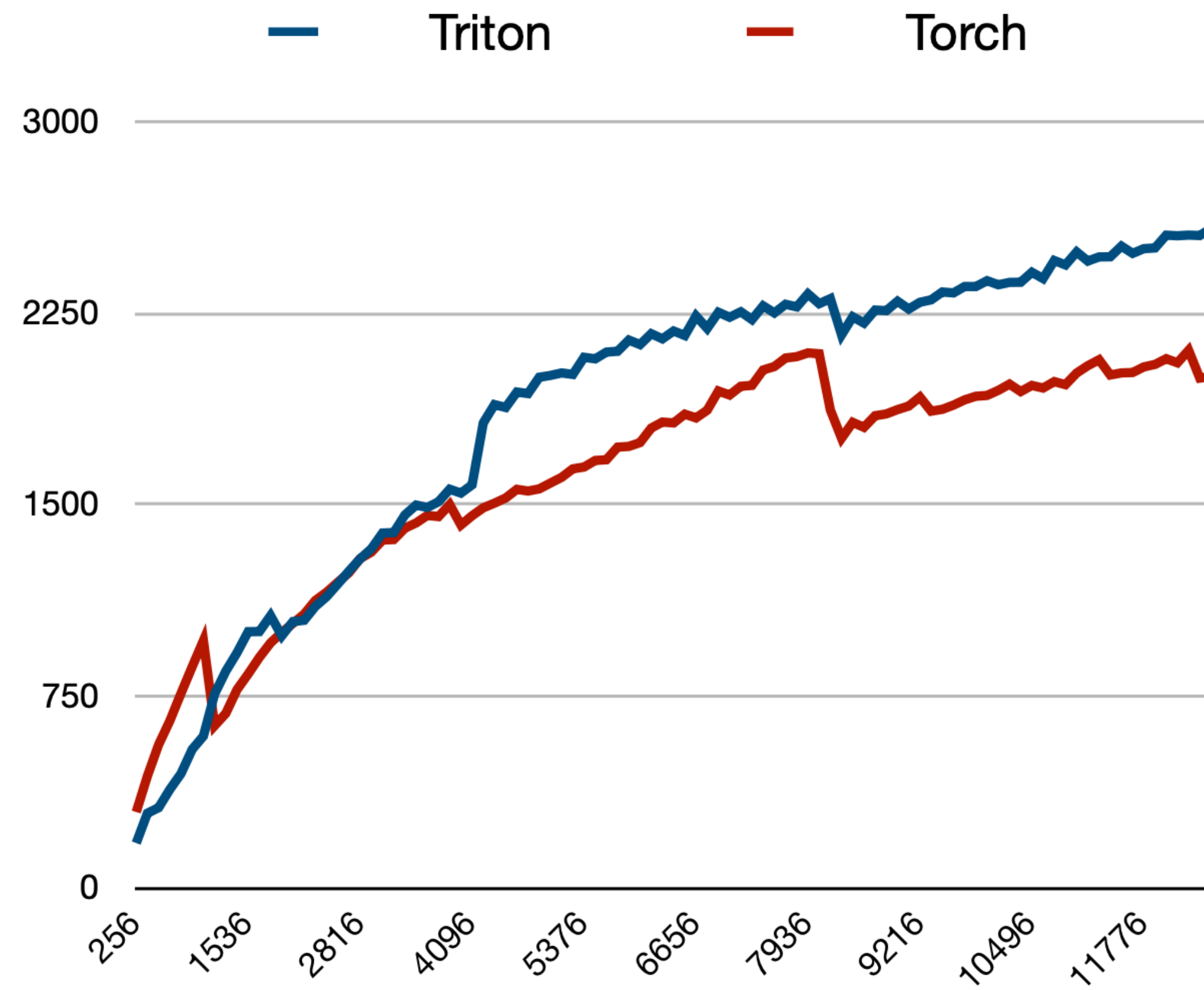
@triton.jit
def _softmax(z_ptr, x_ptr, stride, N, BLOCK: tl.constexpr):
    # Each program instance normalizes a row
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK)

    # Load a row of row-major X to SRAM
    x_ptrs = x_ptr + row*stride + cols
    x = tl.load(x_ptrs, mask = cols < N, other = float('-inf'))

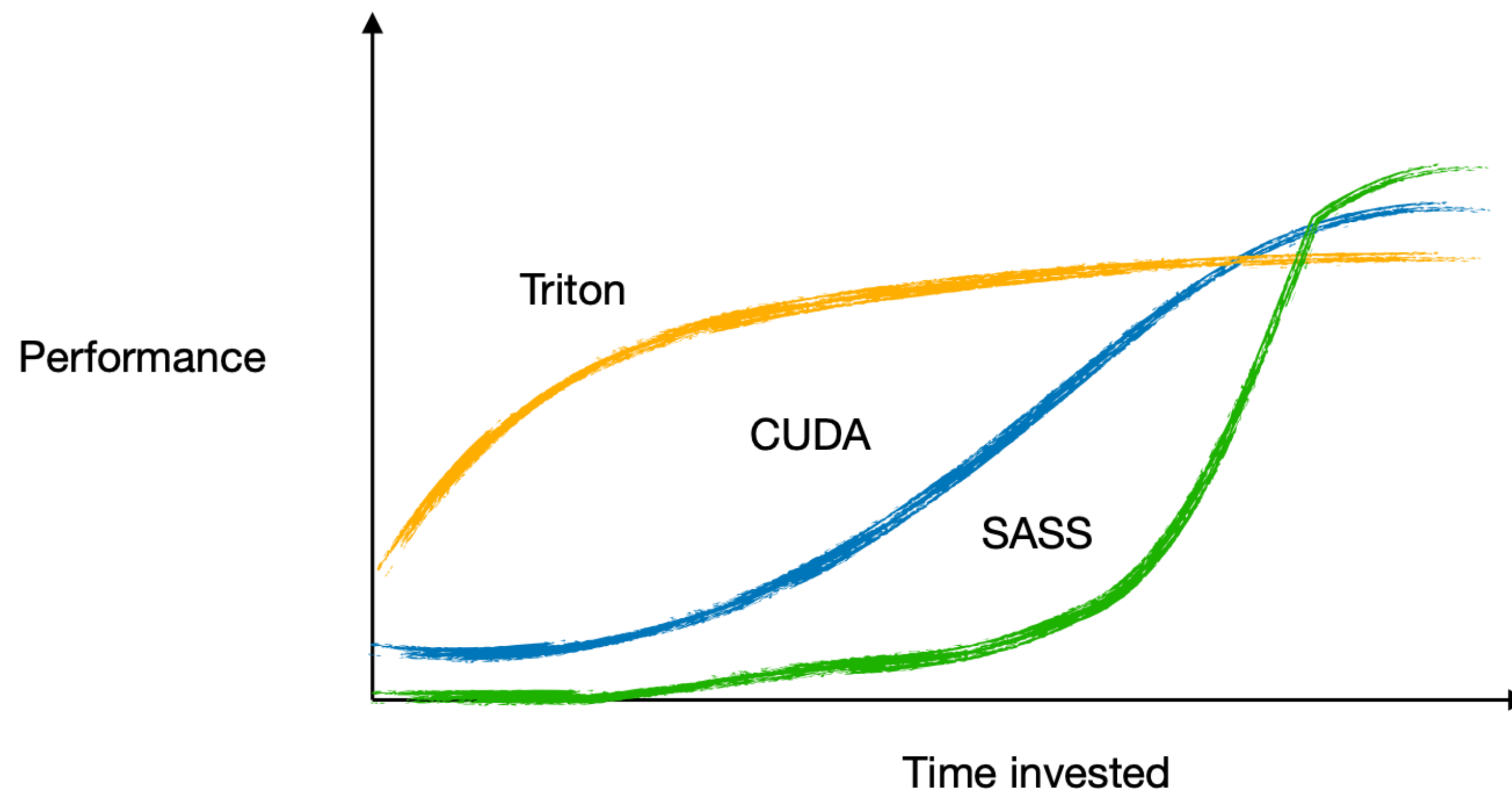
    # Normalization in SRAM, in FP32
    x = x.to(tl.float32)
    x = x - tl.max(x, axis=0)
    num = tl.exp(x)
    den = tl.sum(num, axis=0)
    z = num / den;

    # Write-back to HBM
    tl.store(z_ptr + row*stride + cols, z, mask = cols < N)
```

Performance



Why Triton (seemingly) Succeeds



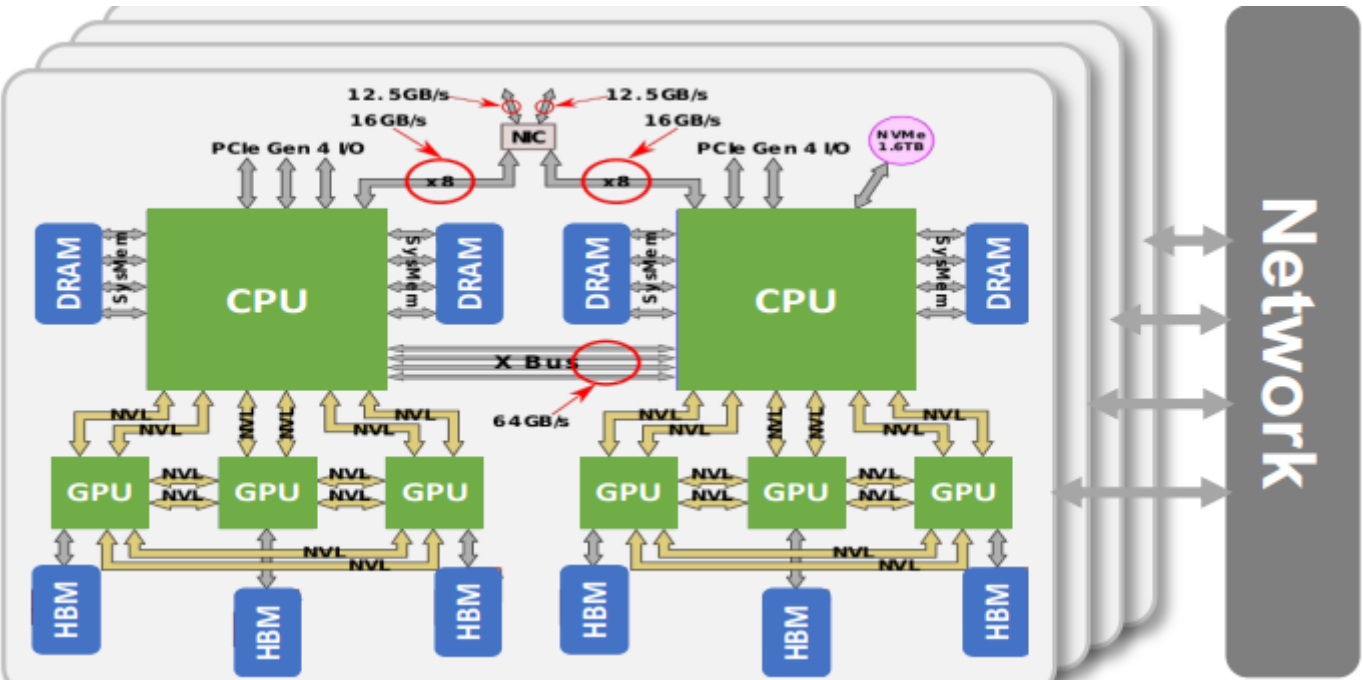
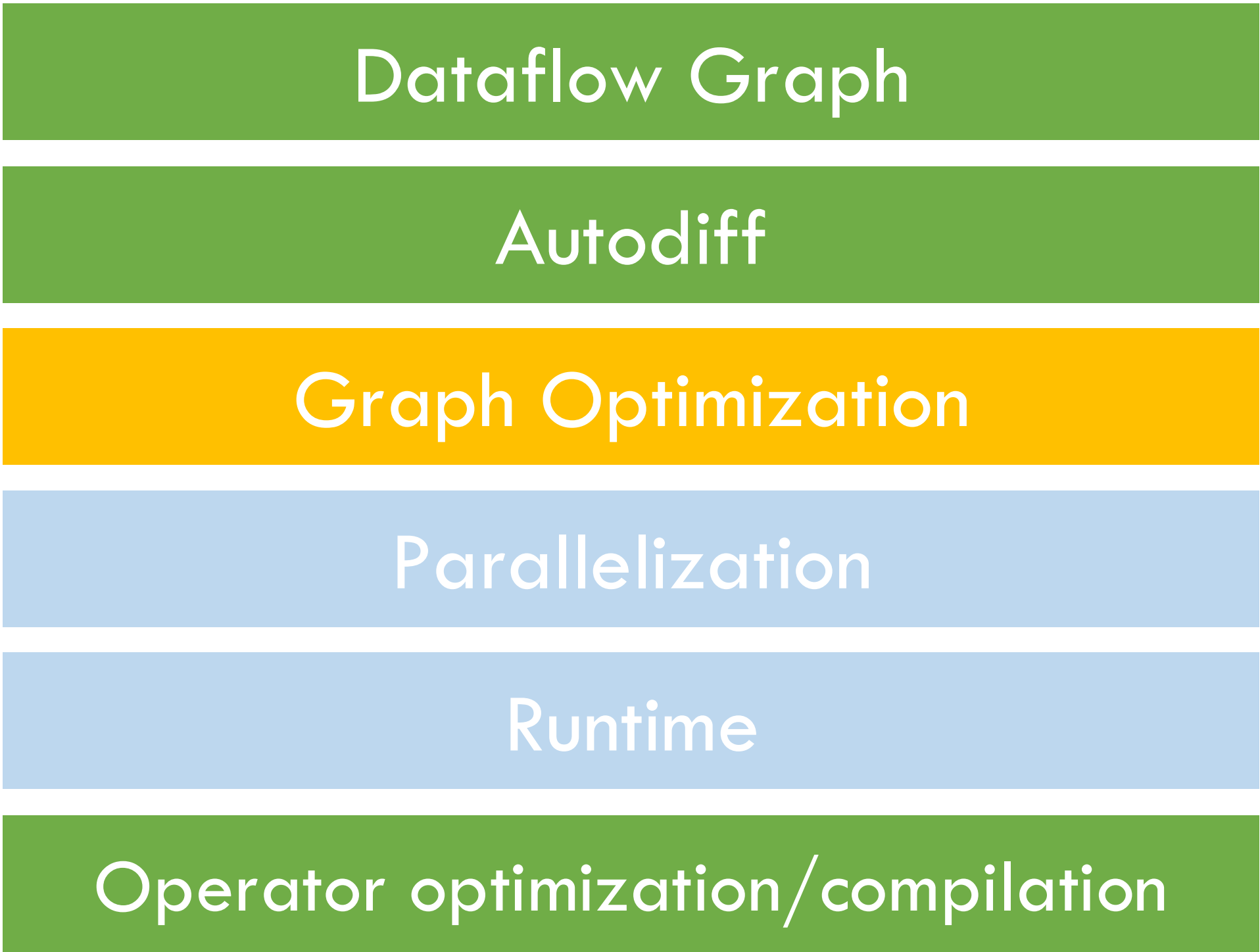
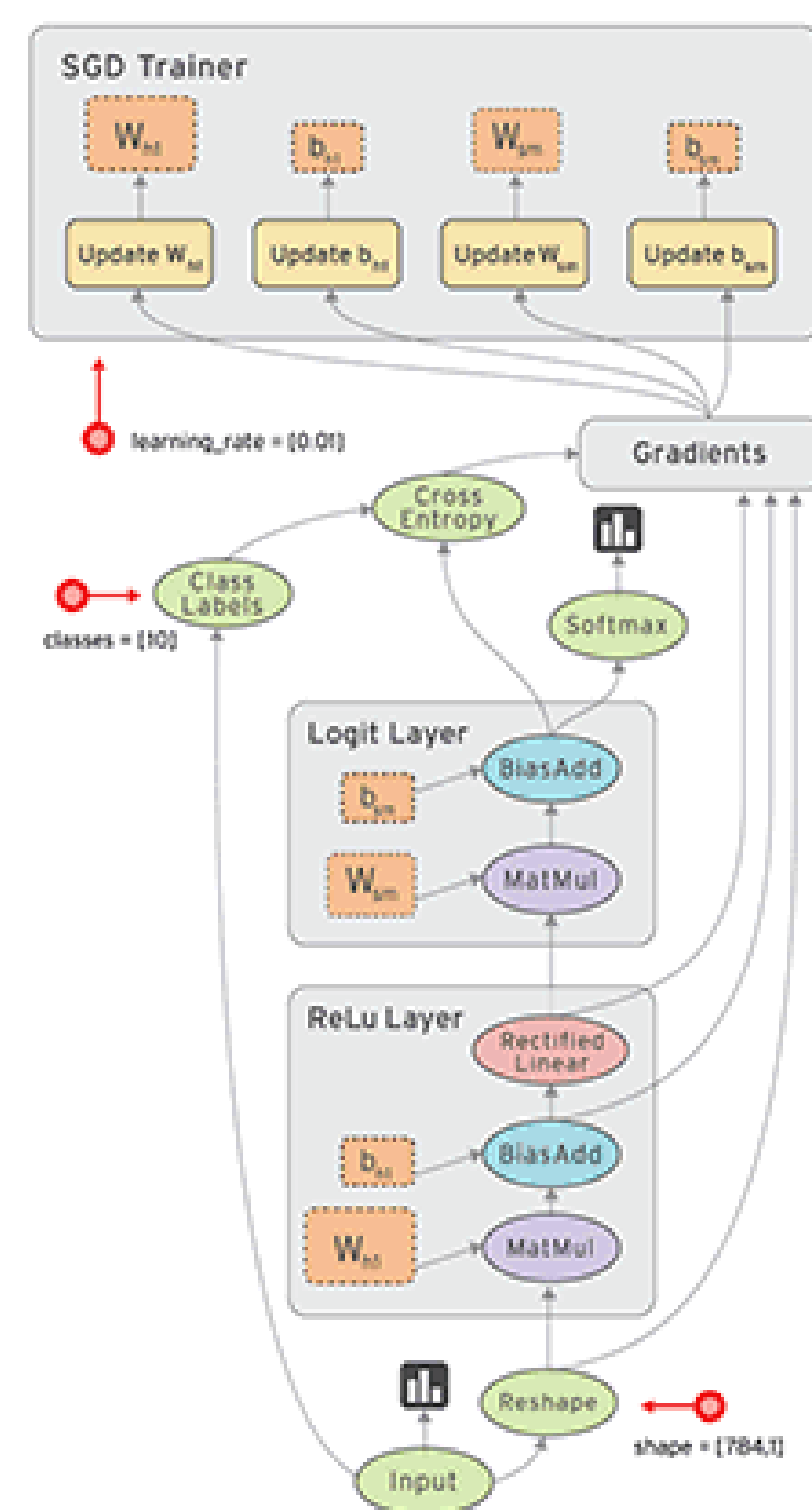
SASS = streaming assembly

Summary: Operator Optimization

Goal: to make individual operator run fast on diverse devices

1. General ways: vectorization, data layout, etc.
2. Matmul-specific: tiling to use fast memory
3. Parallelization SIMD using accelerators
4. Handcrafted operator kernels vs. automatically compile code
5. Triton to find the sweet spot

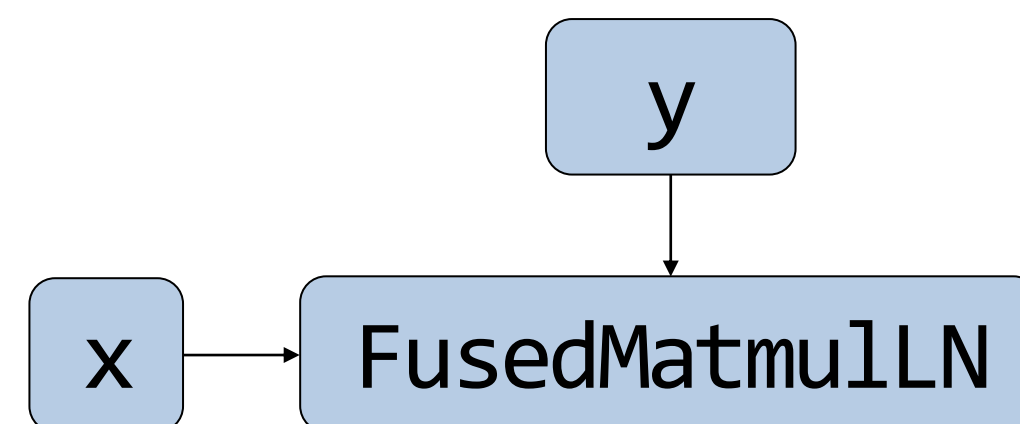
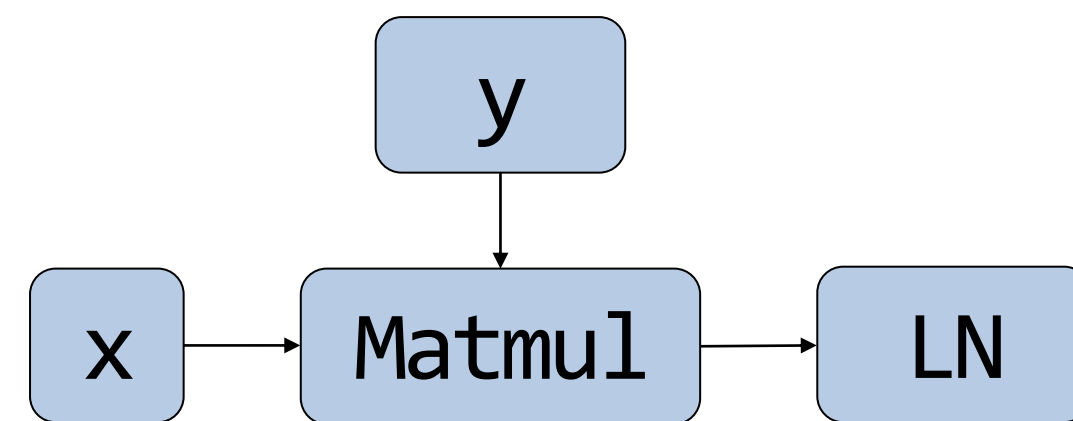
Next: Graph Optimization



Recall Our Goal

- Goal: Rewrite the original Graph G to G' ;
 - G' runs faster than G
 - G' outputs equivalent results
- Straightforward solution: template
 - Human experts write (sub-)graph transformation templates
 - Guarantee correctness and performance gain
 - Run pattern matching over dataflow graph and replace

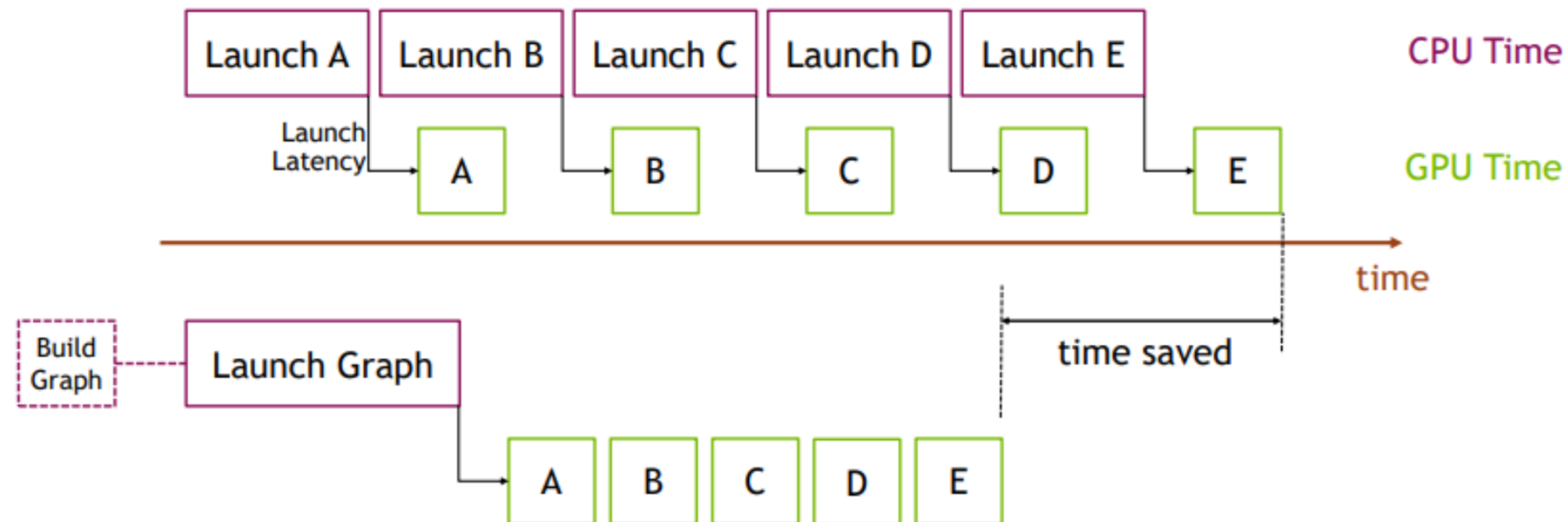
Graph Optimization Templates: Fusion



- Why operator fusion improves performance?
 - Reduce I/O
 - Reduce kernel launching
- Cons:
 - Requiring many fused ops: FusedABCOp
 - At some point, codebase becomes unmanageable

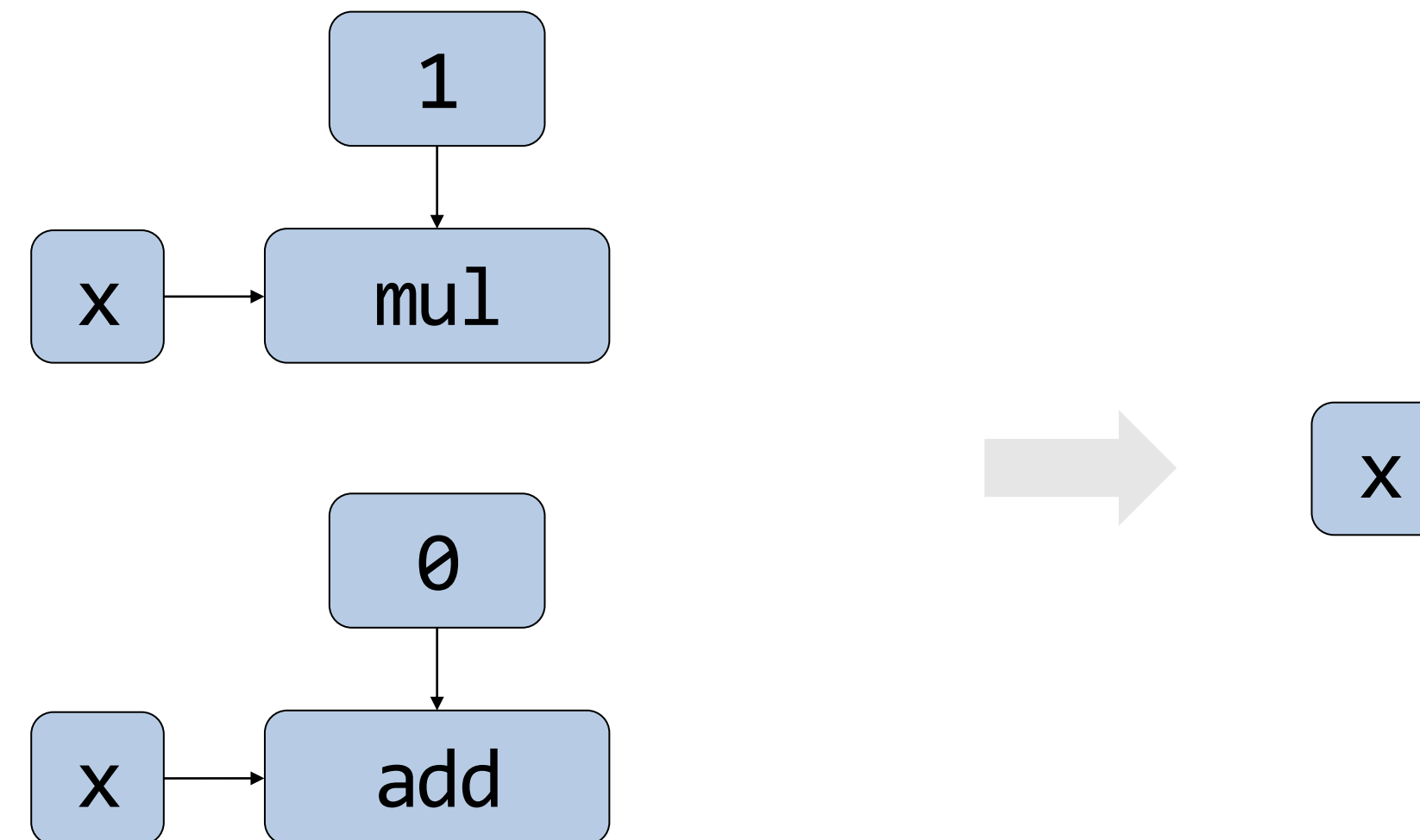
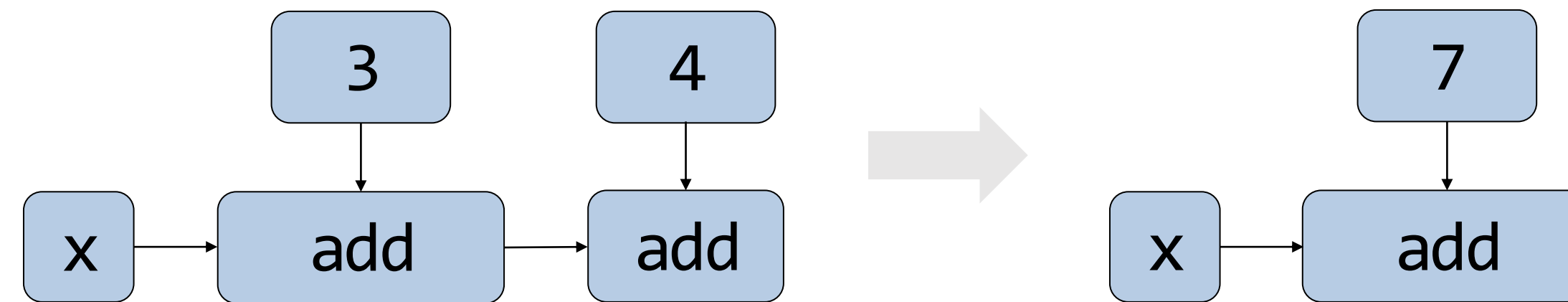
One trade-off in Practice results in “CUDA Graph”

- Users are allowed to program using primitives with high-level APIs
- Graph is captured at CUDA level



More reading: <https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/>

Graph Optimization Templates: Constant Folding



Common Subexpression Elimination (CSE)

$$\begin{array}{l} \dots \\ c = a + b \\ d = a \\ e = b \\ f = d + e \\ d = x \\ \dots \end{array}$$

$$\begin{array}{l} \dots \\ c^3 = a^1 + b^2 \\ d^1 = a^1 \\ e^2 = b^2 \\ \del{f^3 = d^1 + e^2} \\ f^3 = c^3 \\ d^4 = x^4 \\ \dots \end{array}$$

CSE hit

Dead Code Elimination (DCE)

...
 $c = a + b$
 $d = a$
 $e = b$
 $f = d + e$
 $d = x$
....

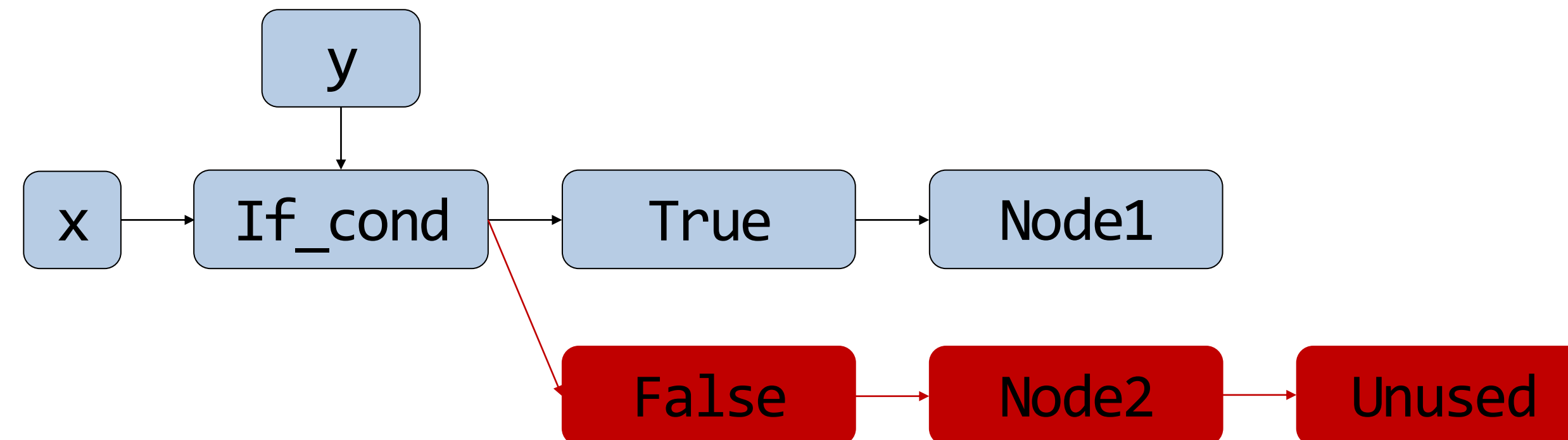
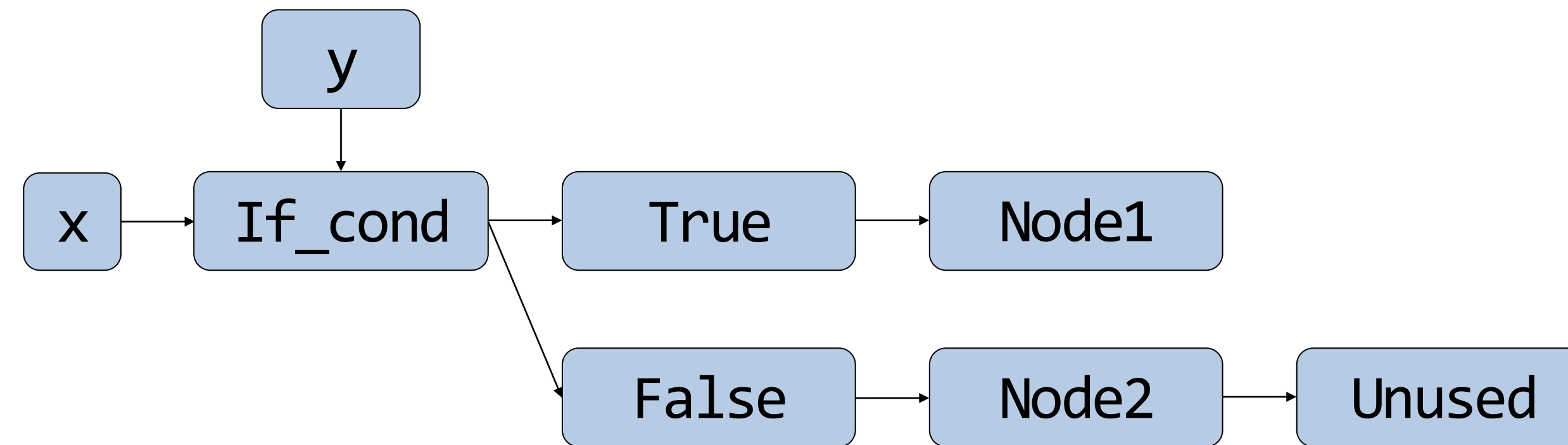
...
 $c^3 = a^1 + b^2$
 $d^1 = a^1$
 $e^2 = b^2$
 ~~$f^3 = d^1 + e^2$~~
 $f^3 = c^3$
 $d^4 = x^4$
....

CSE hit

...
 $c^3 = a^1 + b^2$
 ~~$d^1 = a^1$~~
 $e^2 = b^2$
 ~~$f^3 = d^1 + e^2$~~
 $f^3 = c^3$
 $d^4 = x^4$
....

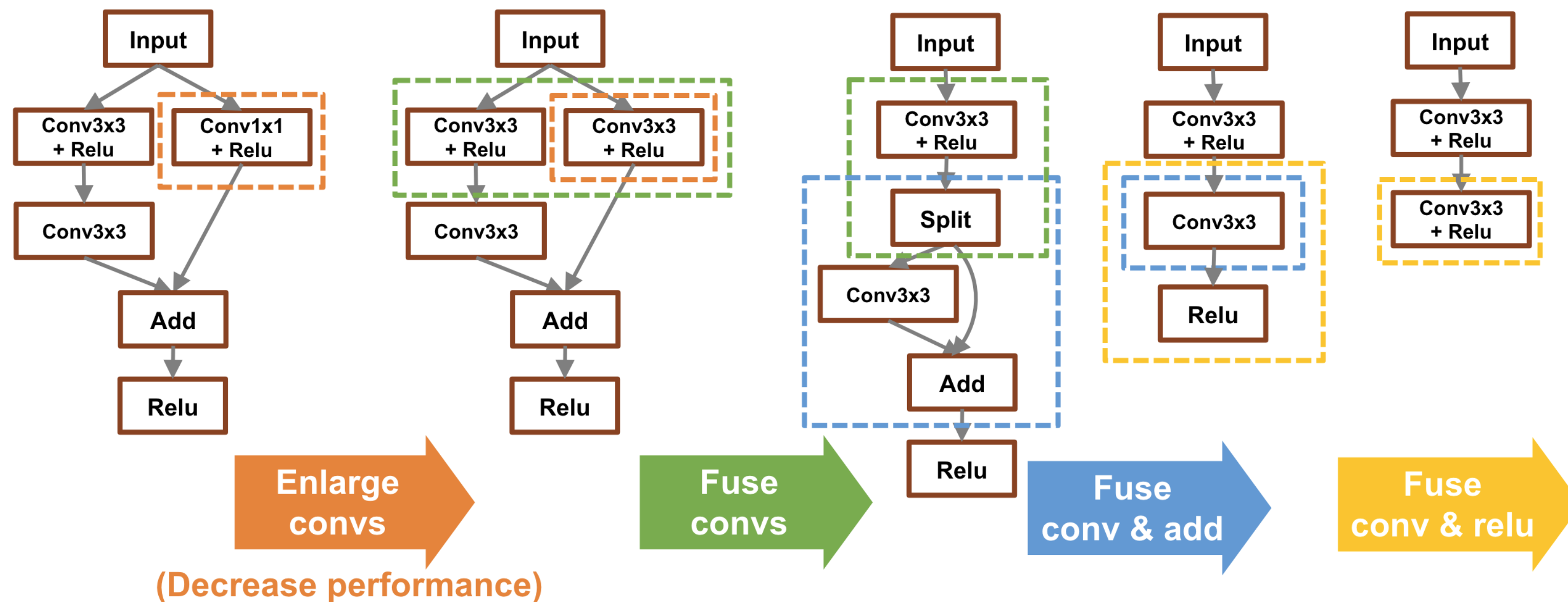
DCE hit

More templates for CSE and DCE



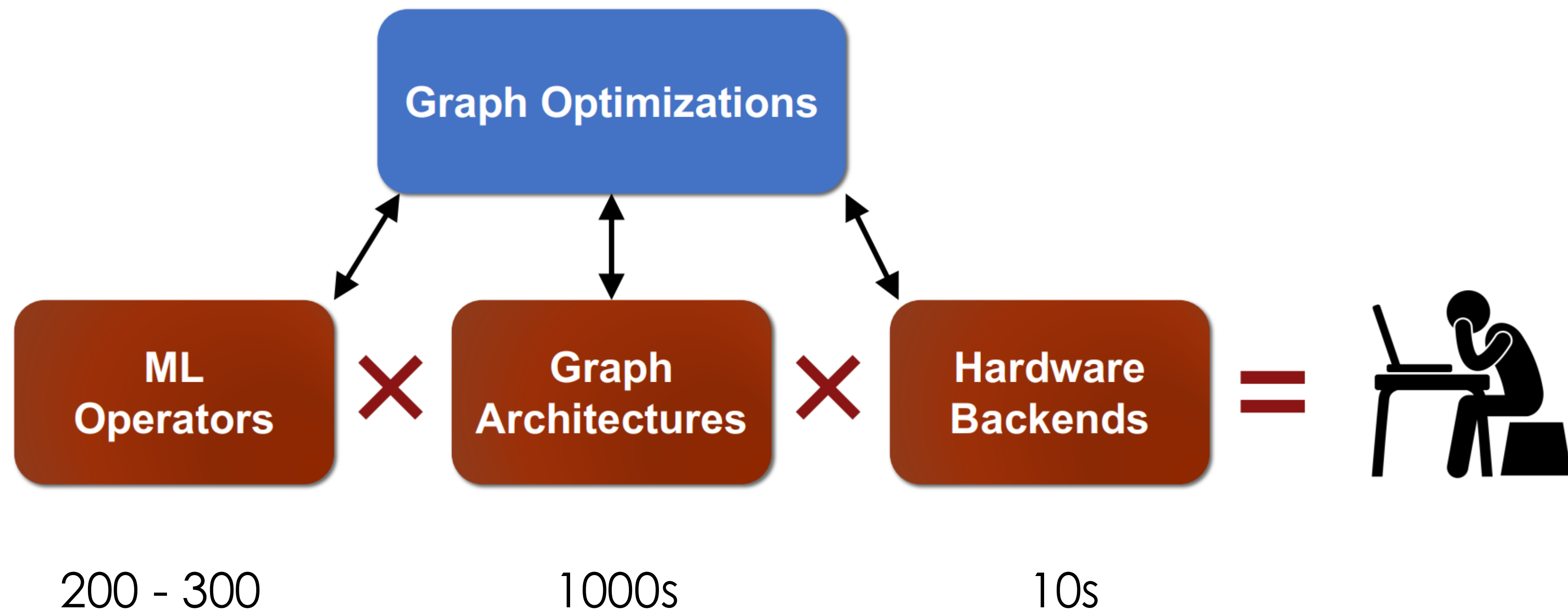
How to ensure performance gain?

- Greedily apply graph optimizations



- The final graph is 30% faster on V100 but 10% slower on K80.

Problems of Template-based Graph Optimizations



Problem: Infeasible to manually design graph optimizations for all cases

Problems of Template-based Graph Optimizations

Robustness

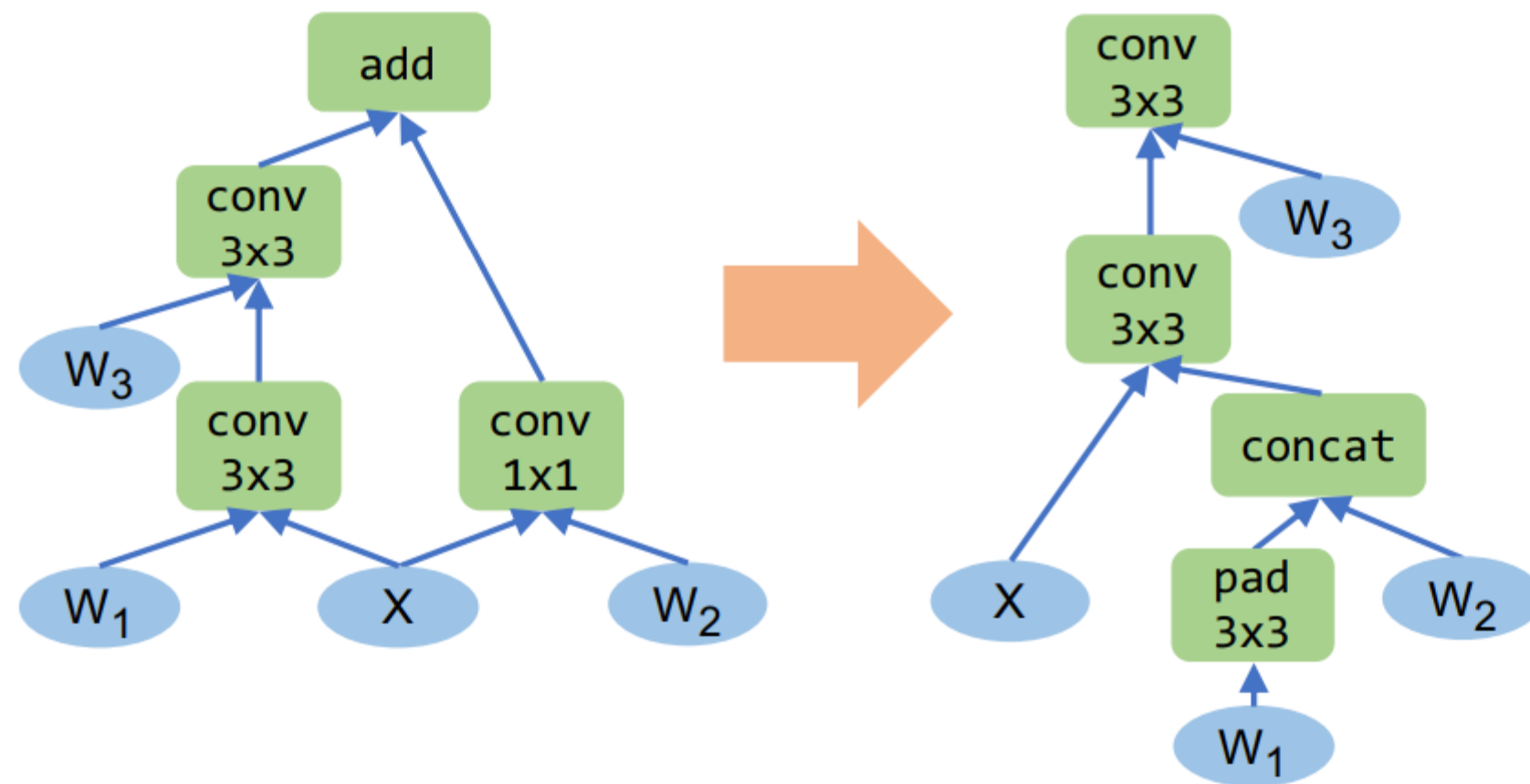
Experts' heuristics do not apply to all DNNs/hardware

Scalability

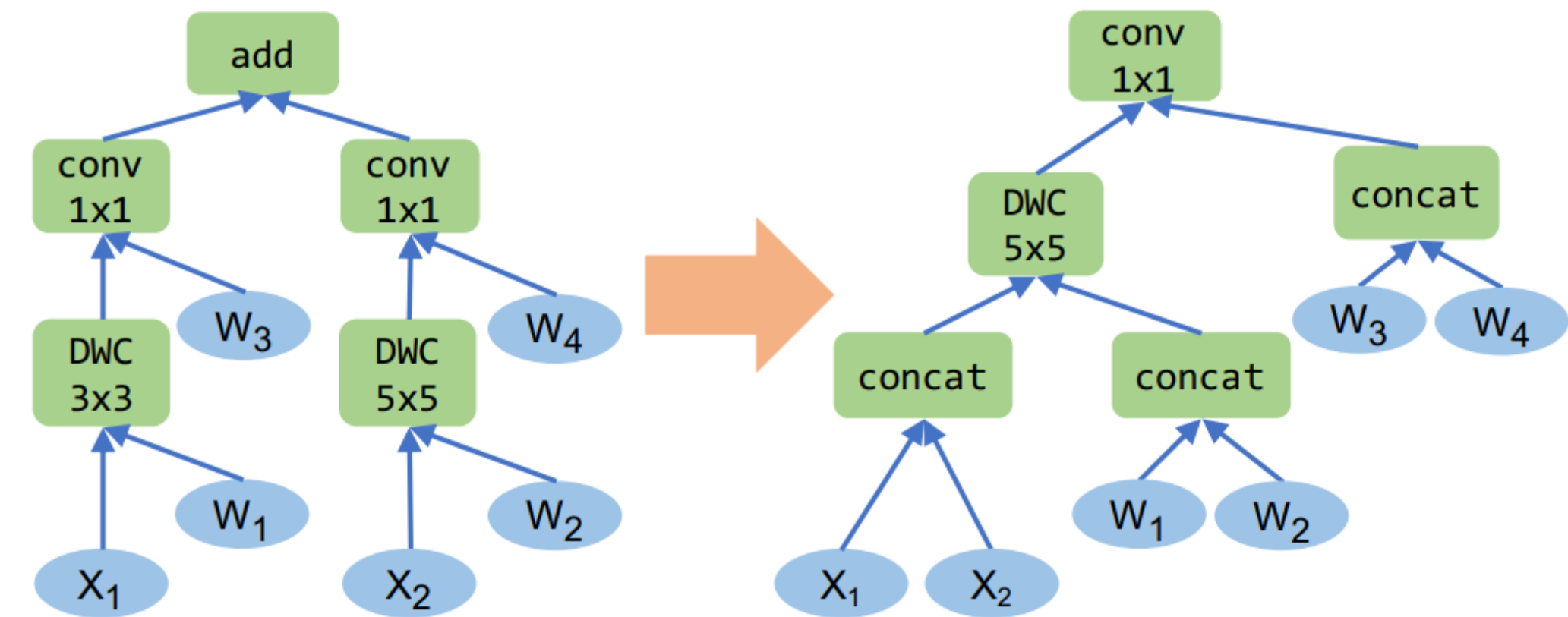
New operators and graph structures require more rules

Performance

Miss subtle optimizations for specific DNNs/hardware



Only apply to **specific hardware**



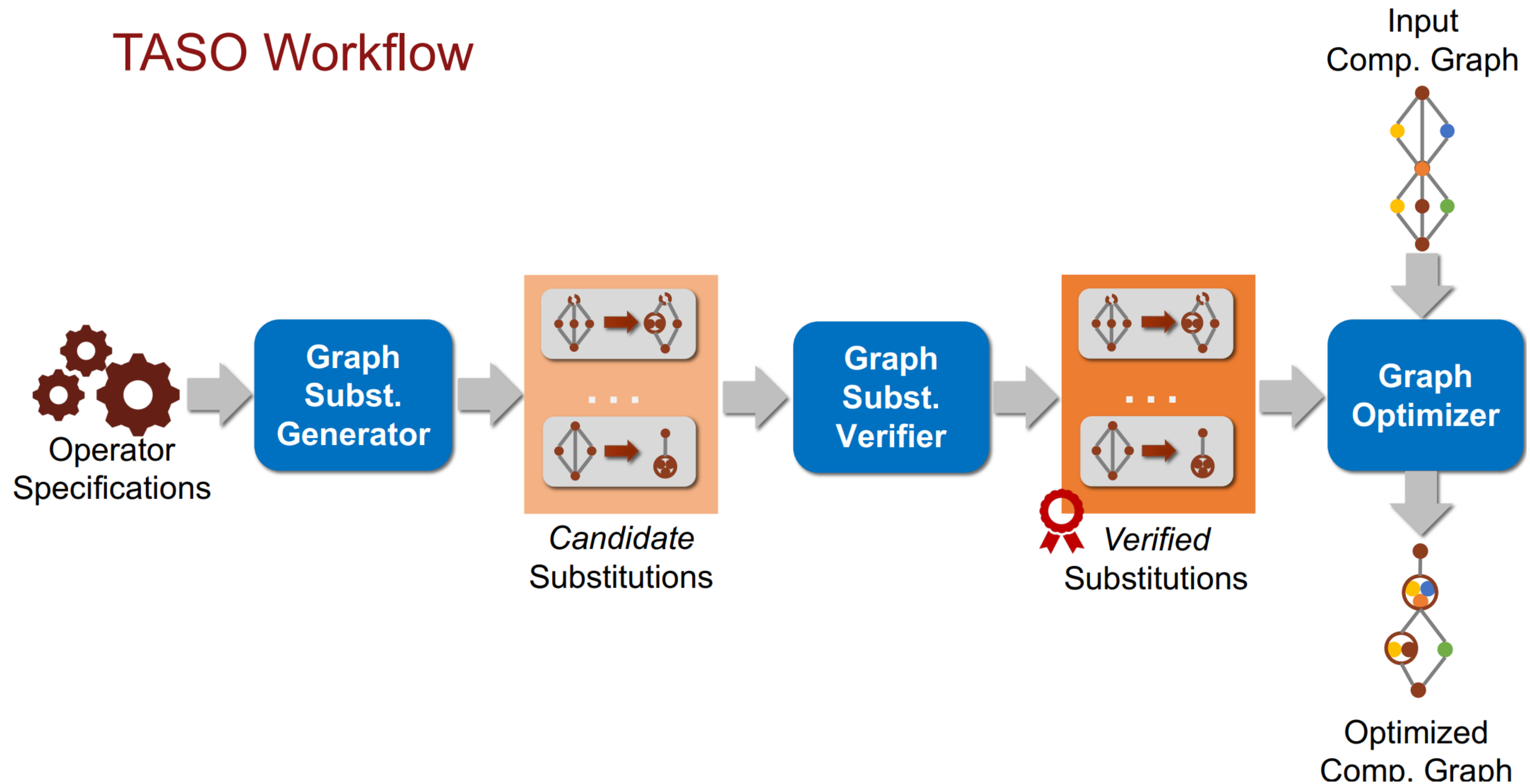
Only apply to **specialized graph structures**

Automate Graph Transformation

Key idea: replace manually-designed graph optimizations with automated generation and verification of graph substitutions for tensor algebra

Enumerate and Verify ALL possible graph

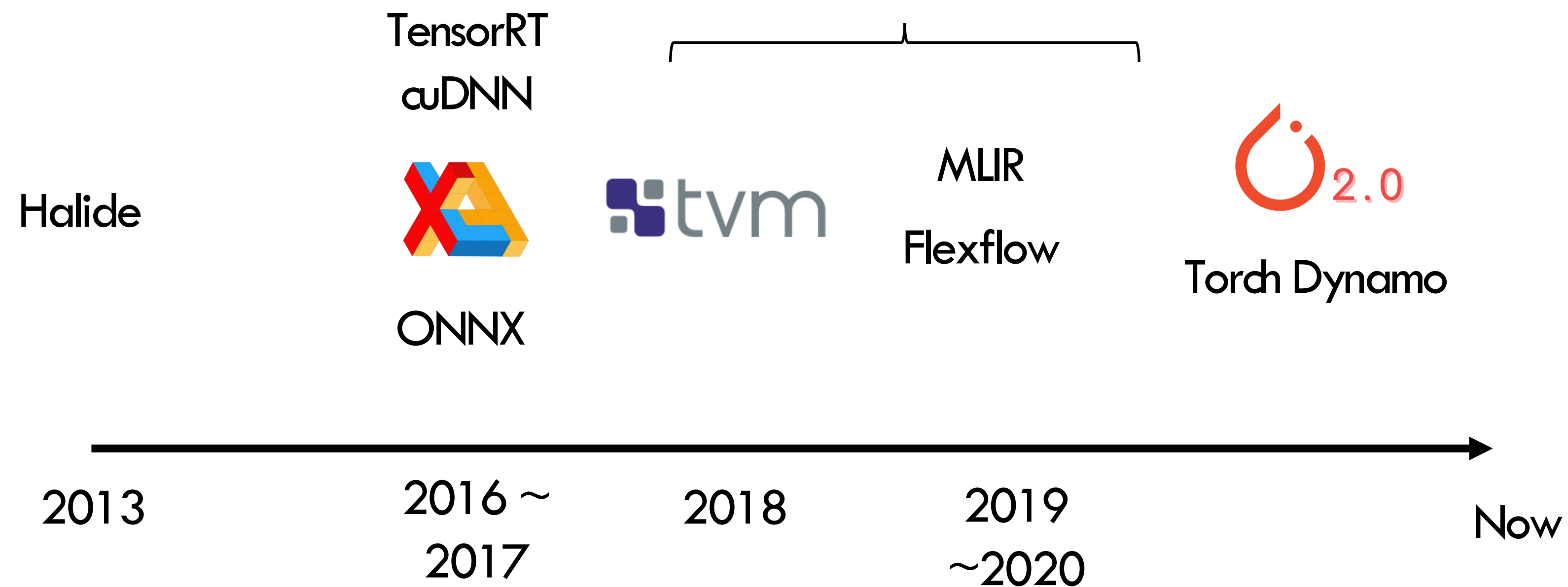
TASO Workflow



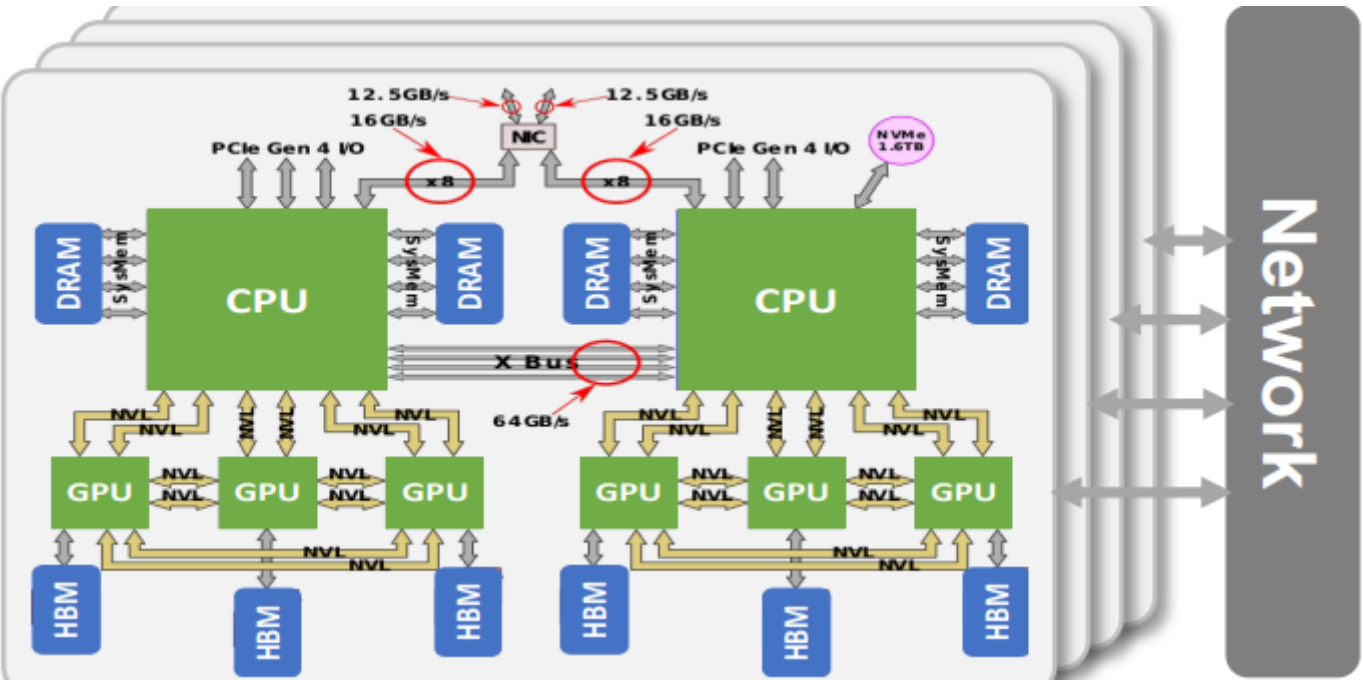
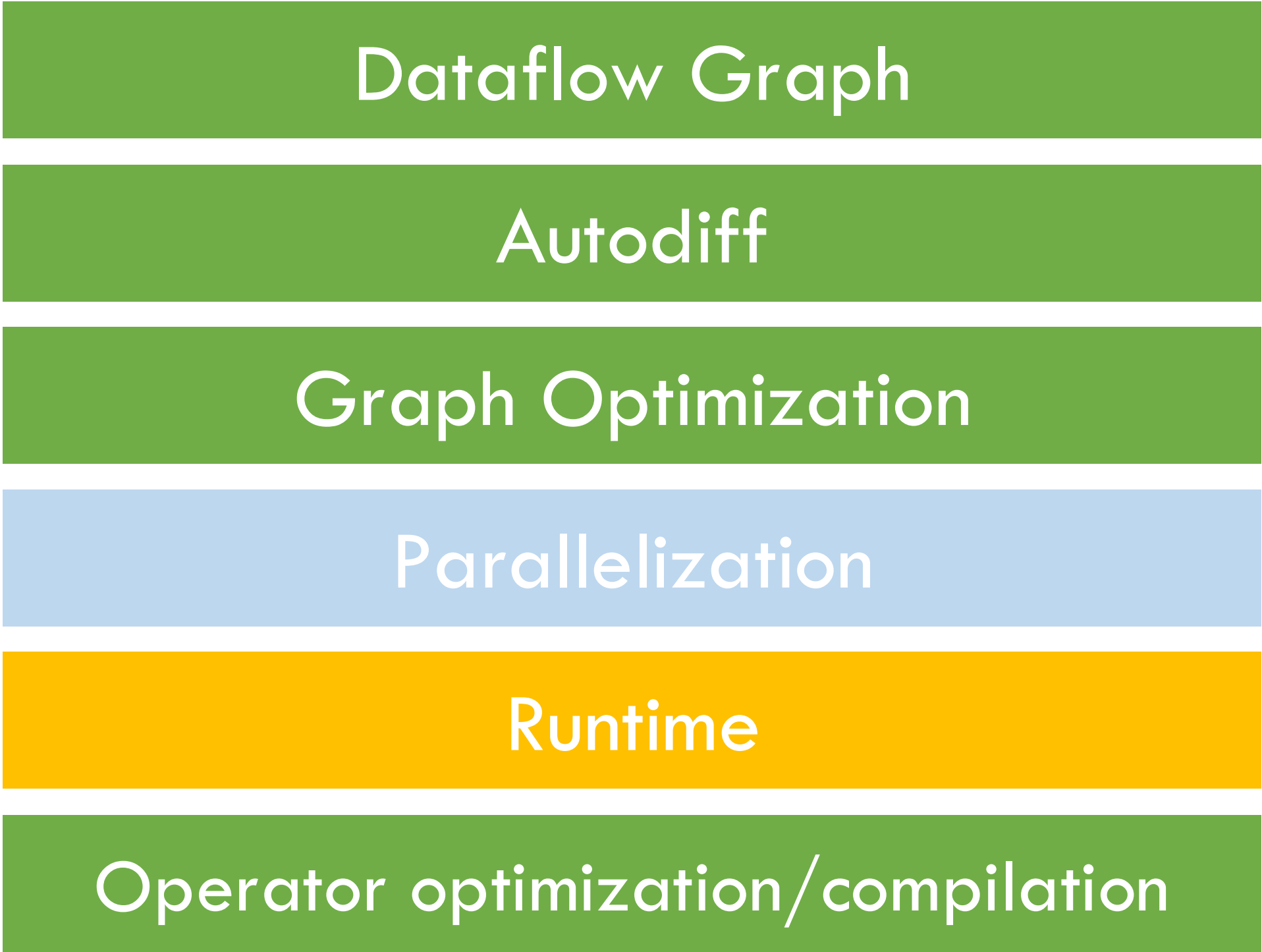
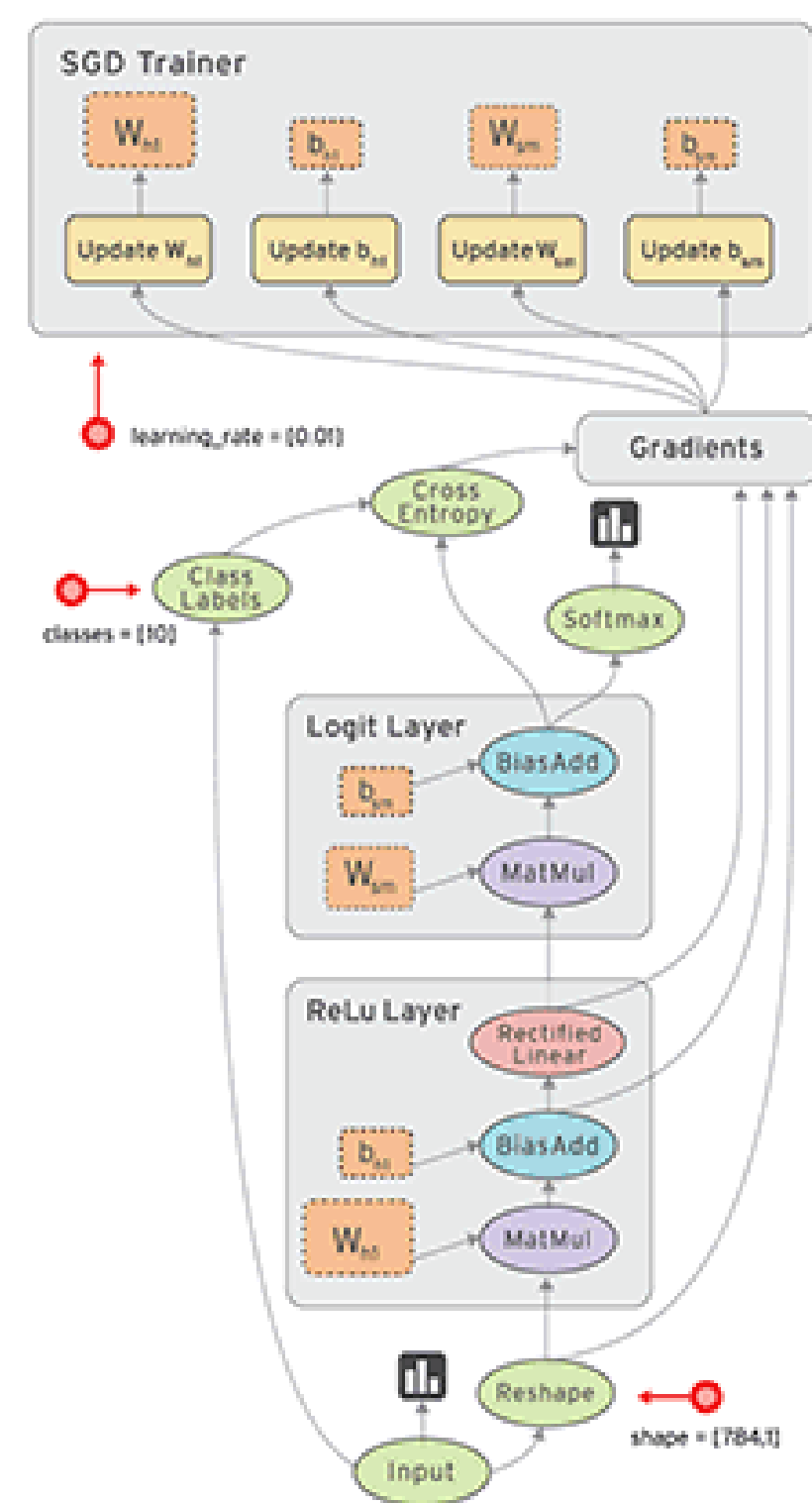
ML Compiler Retrospective

Q: why the community shifts away from compiler

500+ compiler papers are written during



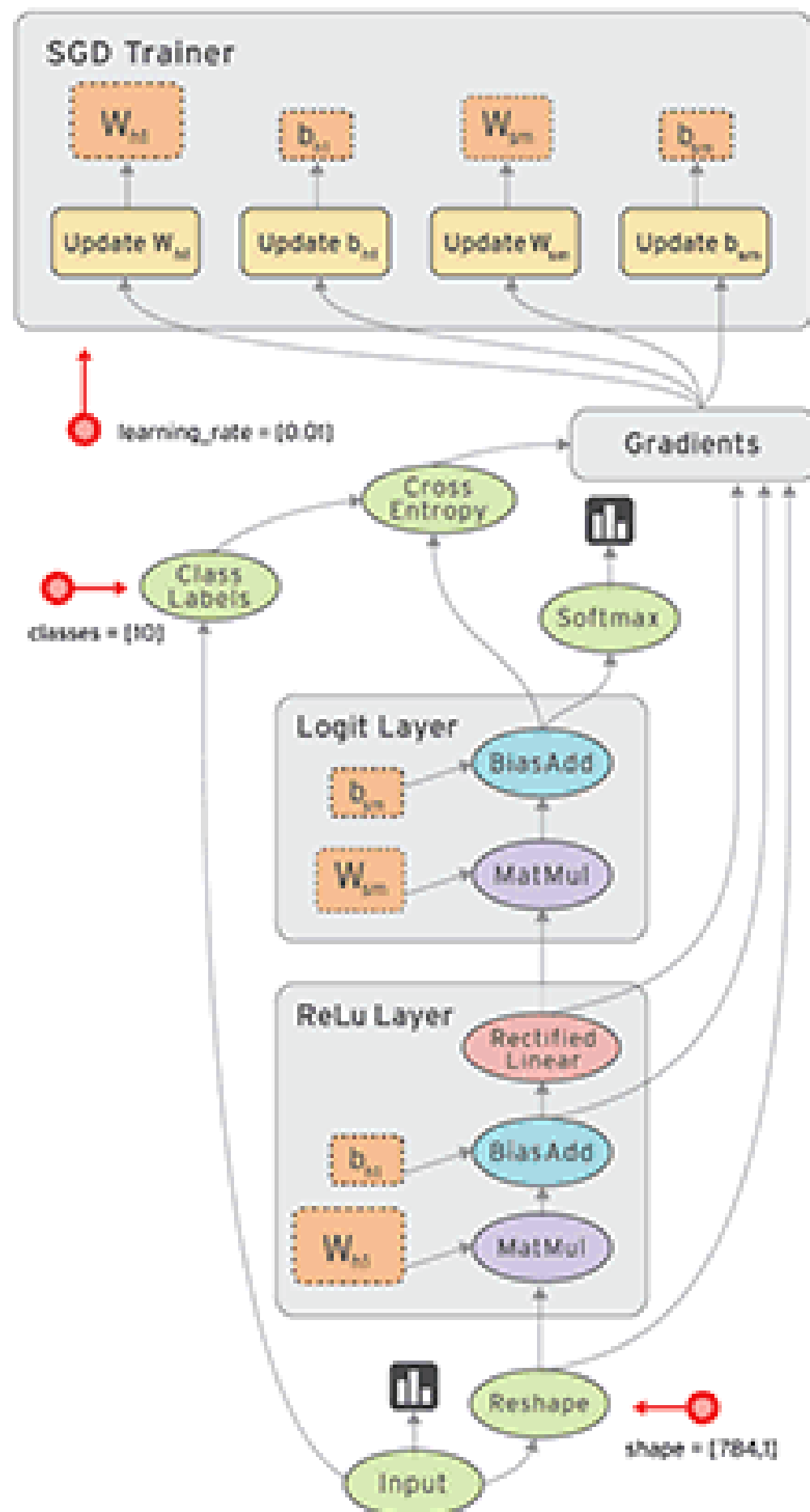
Big Picture: Where We Are



Next: Runtime

- “Batching”
- Checkpointing and rematerialization
- Swapping
- Quantization, Mixed precision, and Pruning

Run the graph: gradient descent with minibatches



for $t = 1 \rightarrow N$

$$\theta^{(t+1)} = f(\theta^{(t)}, \nabla_L(\theta^{(t)}, D^{(t)}))$$

Disambiguate the Term “Batch”

- In deep learning training
 - Batch
 - Mini-batch
 - Micro-batch

In optimization and gradient descent:

- Full-batch gradient descent
- stochastic gradient descent
- In big data processing
 - Batch/offline processing
 - Streaming/online processing

Disambiguate the Term “Batch”

Term	Context	Meaning
Batch	Deep Learning	A group of training samples processed together
Mini-Batch	Deep Learning	A smaller subset of a dataset used per training iteration
Micro-Batch	Deep Learning	A split of a mini-batch, often used for pipeline parallelism
Batch Processing	Big Data	Processing large datasets in bulk
Micro-Batching	Big Data	Processing small groups of events in short time windows

Next: Learning Goals

- Memory and Scheduling
 - Checkpointing and rematerialization
 - Swapping
- Memory and Compute
 - Quantization
 - Mixed precision

Dataflow Graph

Autodiff

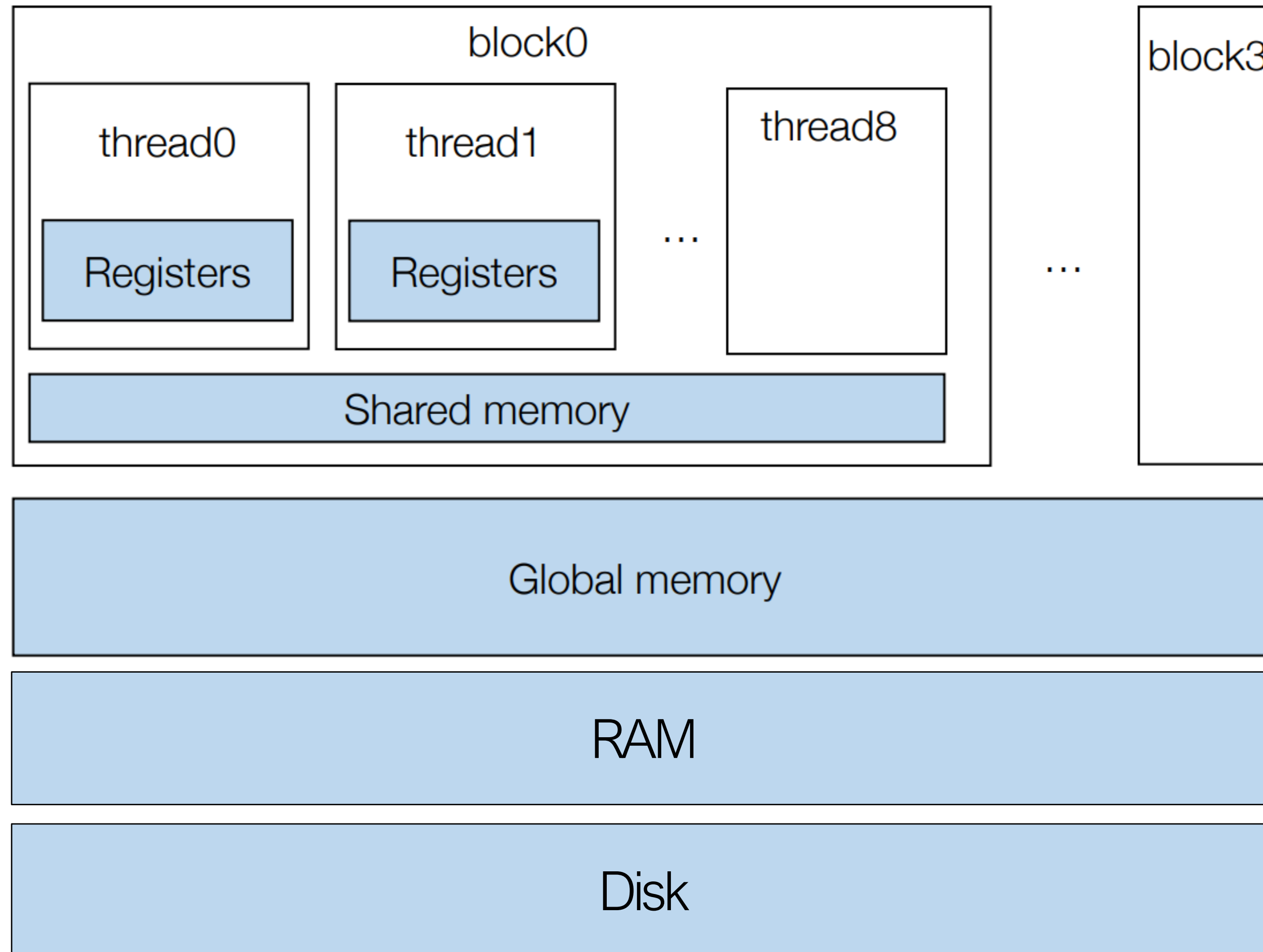
Graph Optimization

Parallelization

Runtime

Operator

Recall: data movement path



Shared memory: 64 KB per core

GPU memory(Global memory):

RTX3080 10GB

RTX3090 24GB

A100 40/80 GB

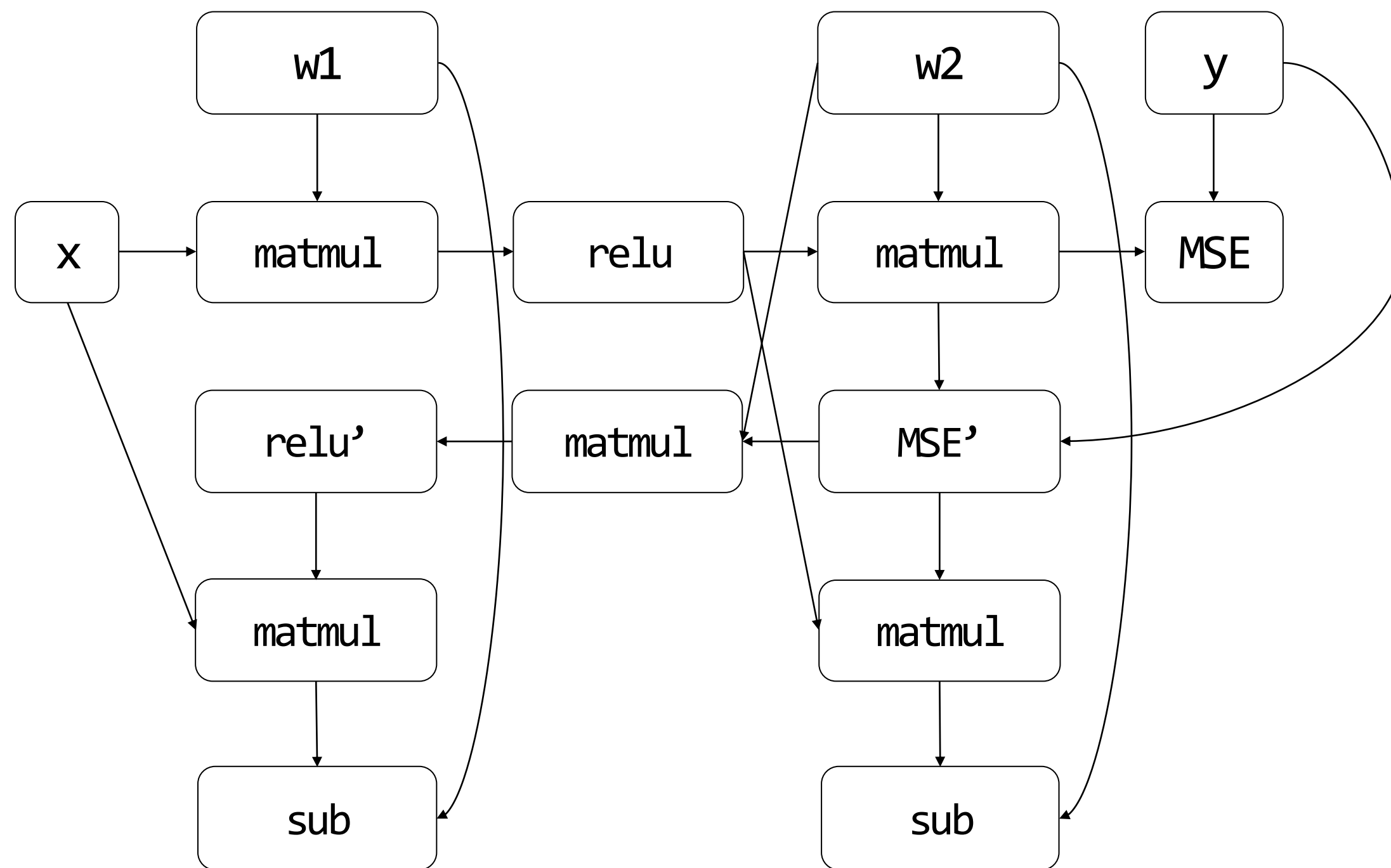
Our Goal on Memory

- Fit the workload on limited memory and ensure
peak memory < available memory

Note:

- Normally, we do not need to min (memory)
- We do not need to min(max(memory))
- We just need $\max(\text{memory}) < \text{available memory}$
 - Unless otherwise specificized

Source of Memory Consumption



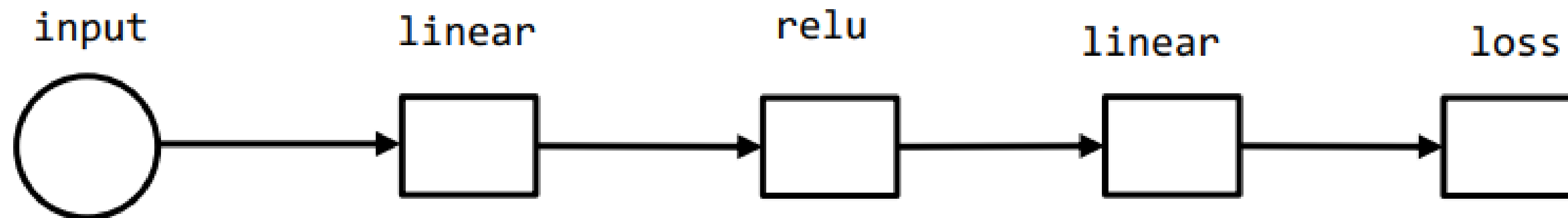
Sources of memory consumption

- Model weights
- Intermediate activation values
- Optimizer states

Methods to Analyze Memory

- Size
 - How large the memory is
- Lifetime:
 - When will this memory be needed and when not

Start with Inference: Lifetime?

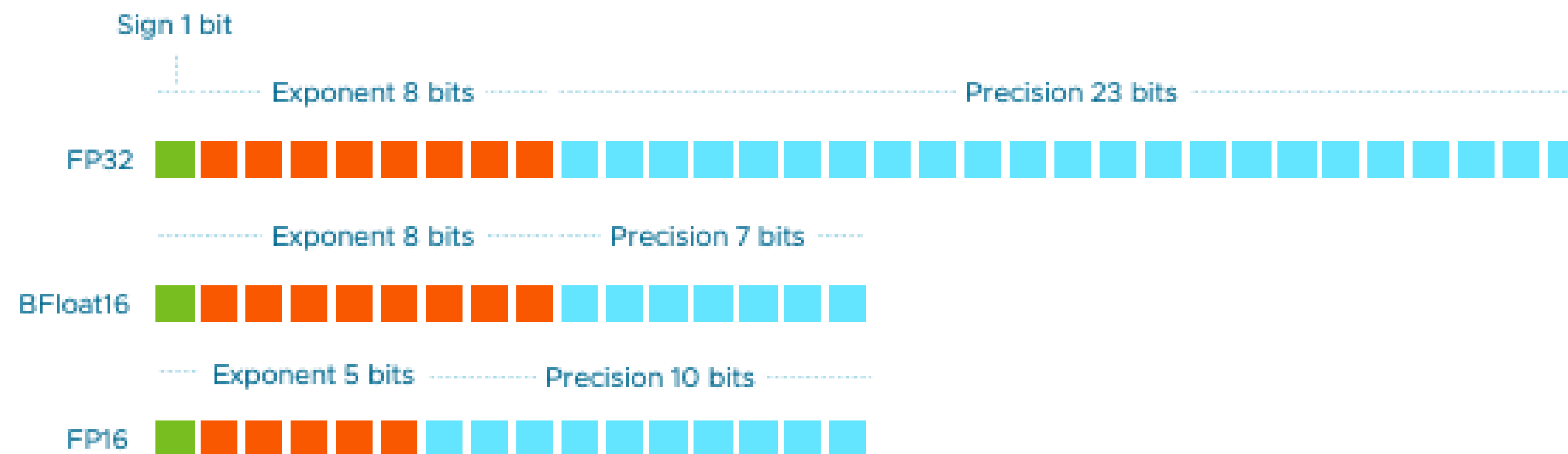


We only need $O(1)$ memory for computing the final output of a N layer deep network by cycling through two buffers

Lifetime of

- weights
- activations?
- Optimizer states?

Estimate size: Popular floating point standards



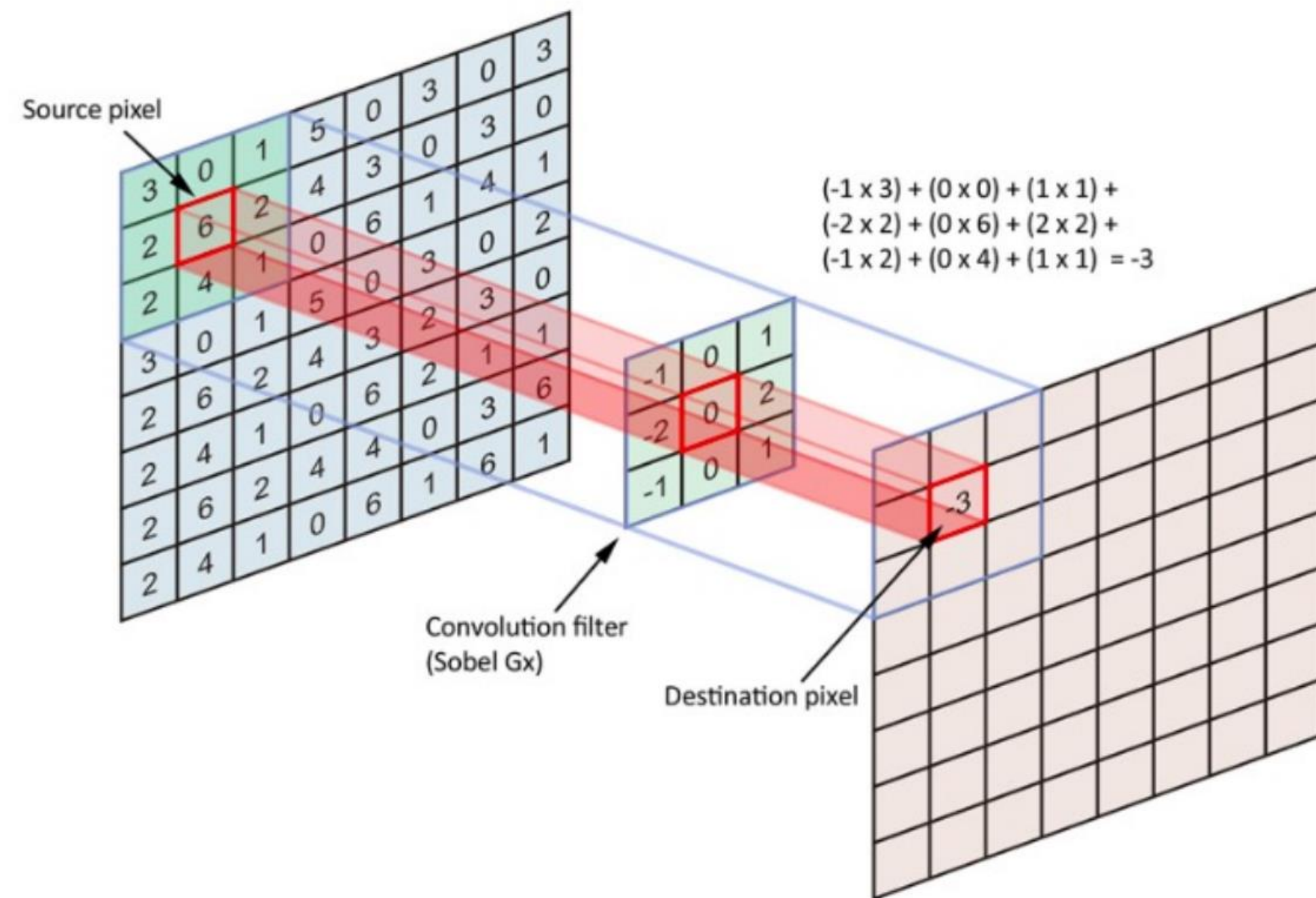
- What does **exponent** and **fraction** control in float point representation?
- What's the difference between bf16 and fp16?
- Will come back to this in detail later

Estimate the weight size: GPT-3 as an example

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

- Model weights: 175B, each param = 16 / 32 bits = 2 / 4 bytes
 - $175\text{B} * 2 / 4 = 350\text{G} / 700\text{G}$
 - Rule of thumb: check precision, and $N*2$ (16bits) or $N*4$ (32bits)

Estimate the activation size



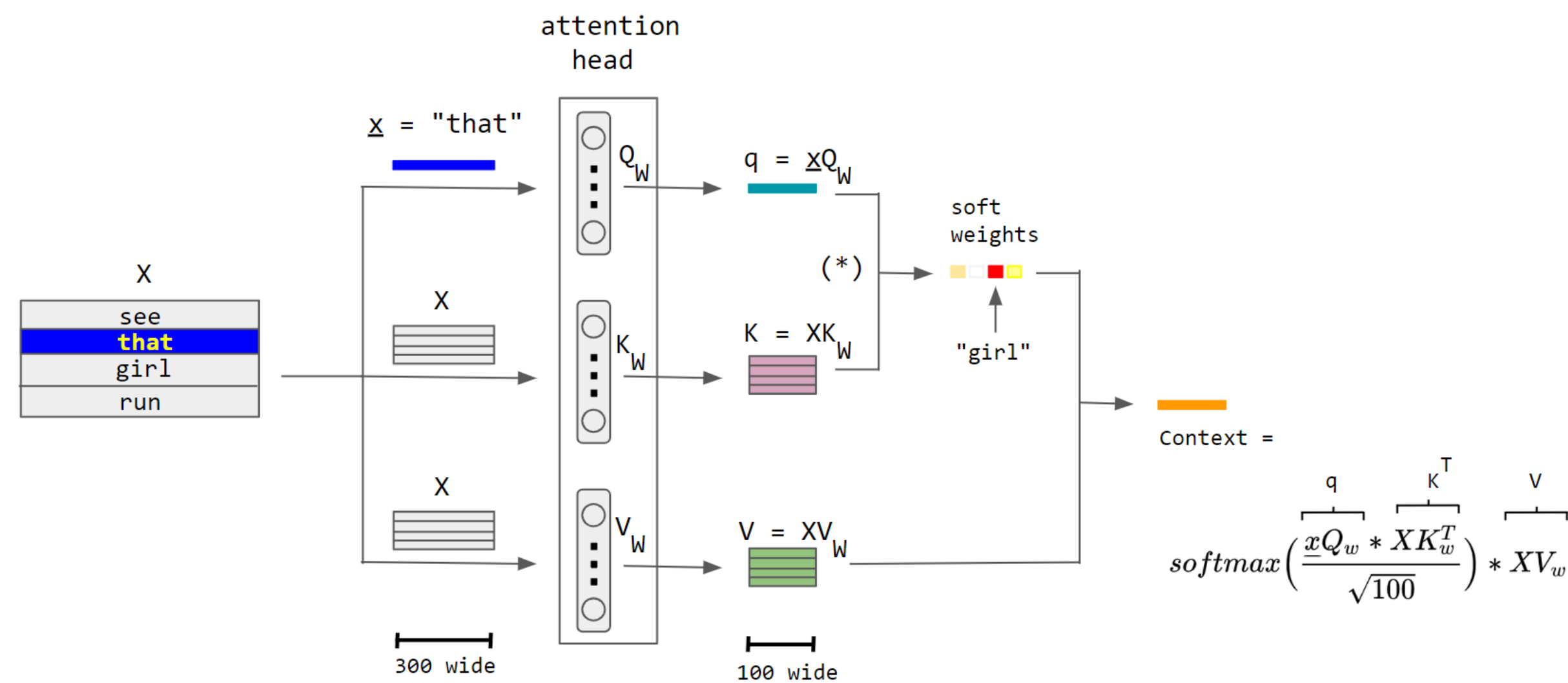
- Conv2d activation:
 - Input: bs, nc, wi, hi
 - Output: bs, nc, wo, ho
 - Activation size:
 - $bs * nc * wo * ho * \text{sizeof}(\text{element})$

Estimate the activation size: MLP

$$C = X * W$$

- Matmul activation
 - Input: $bs * m * n$
 - Output: $bs * m * p$
 - Activation size:
 - $bs * m * p * \text{sizeof}(\text{element})$

Estimate the activation size: Transformers



- transformer activation
 - Input: $bs * h * seq_len$
 - Output: $bs * h * seq_len$
 - Activation size:
 - $bs * h * seq_len * \text{sizeof}(\text{element})$

Estimate the activation size: Transformers

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

GPT-3 Transformers per-layer activation (assuming seq_len = 1):

- $\text{bs} * \text{seq_len} * d_{\text{model}}: 3.2\text{M} * 12288 = 39.321\text{B} = 78 / 156 \text{ G}$
- Note: For each operator inside a transformer layer, we also have activations. We'll come back to this later.

Estimate the Optimizer State Size: Adam

- Adam Optimizer: What is the memory added?

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

Optimizer state: first
moment estimate

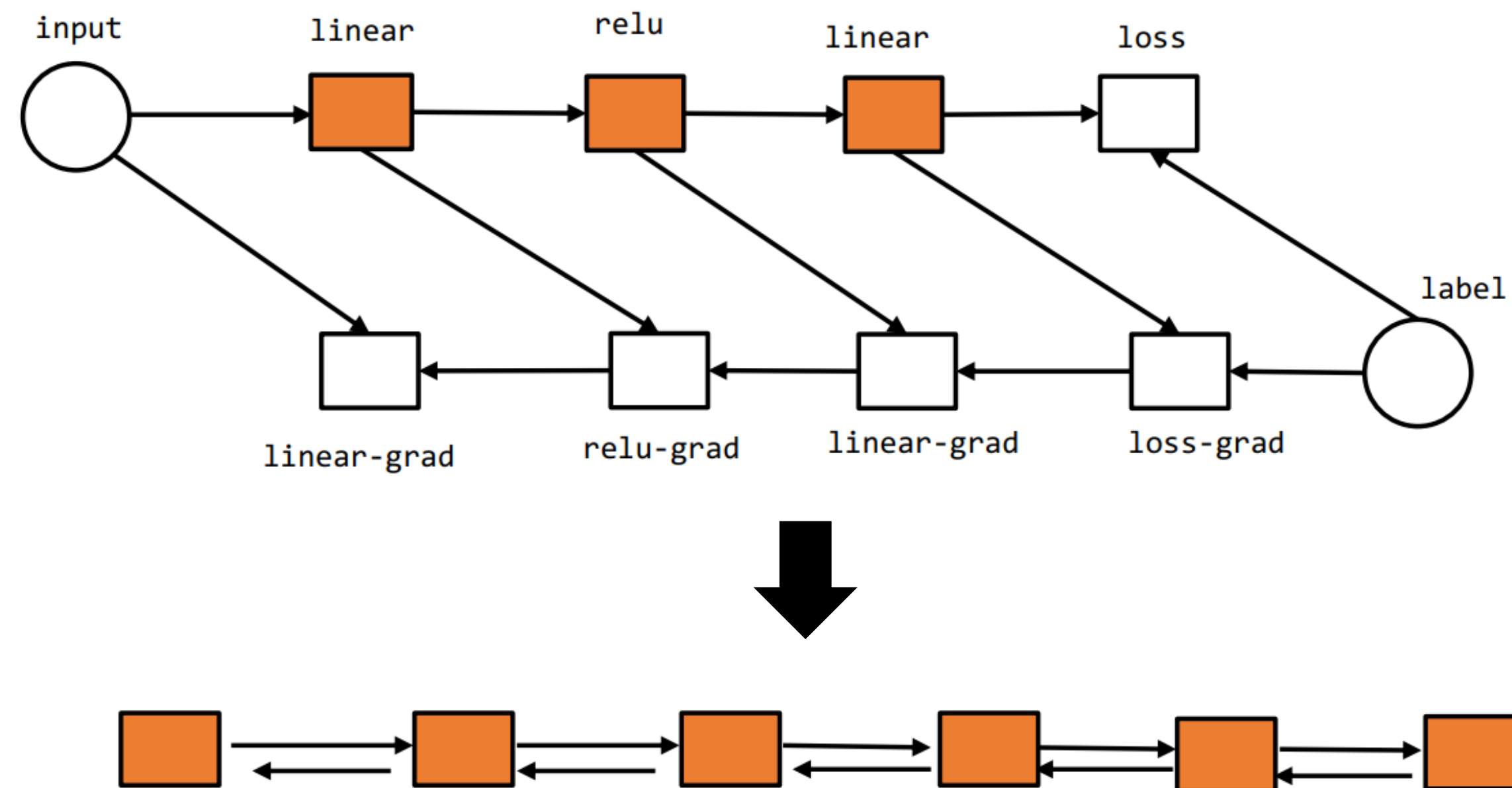
(mean)

Optimizer state: second
moment estimate
(variance)

Estimate the Optimizer State Size?

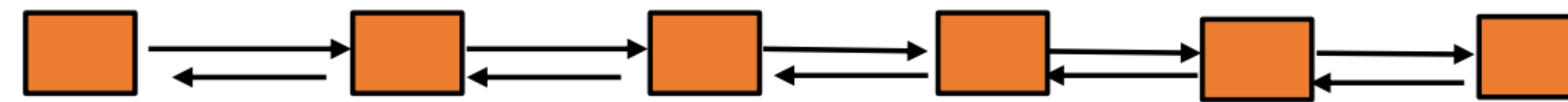
- Gradient w.r.t. parameters: $N * \text{sizeof}(\text{element})$
- First moment: $N * \text{sizeof}(\text{element})$
- Second moment: $N * \text{sizeof}(\text{element})$
- In total:
 - $3N * \text{sizeof}(\text{element})$

Lifetime of activations at training



- Because the need to keep intermediate value around for the gradient steps. Training a N-layer neural network would require $O(N)$.

Memory Overview



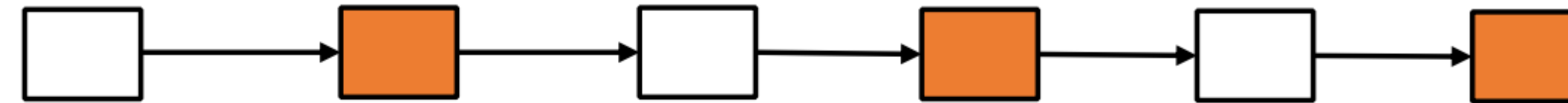
- Parameters: $175\text{B} * (\text{fp32}) = 350 / 700 \text{ G}$
- Activations:
 - At the transformer boundary: $(N = 96) * 78 / 156 \text{ G} = 7488 / 14976 \text{ G}$
 - (This is not accurate because transformers is a composite layers.)
 - Optimizer states: $(\text{precision: fp32}) * 3 * 175\text{B} = (12) * 175 \text{ G}$

Reduce memory

- Single Device (today)
- Parallelization (later)

Reduce activation memory

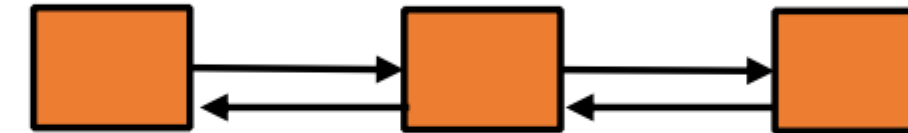
Step 0:



Step 1:



Step 2:

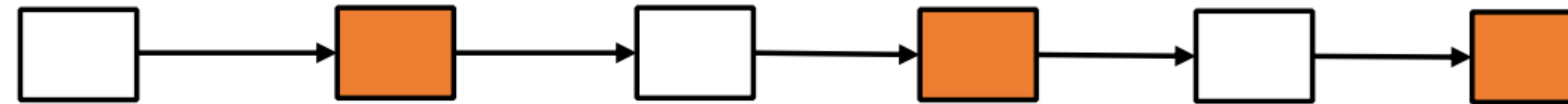


Idea:

- The activation is not needed again until the backward pass comes
- Discard some of them and recompute the missing intermediate nodes in small segments

Reduce activation memory

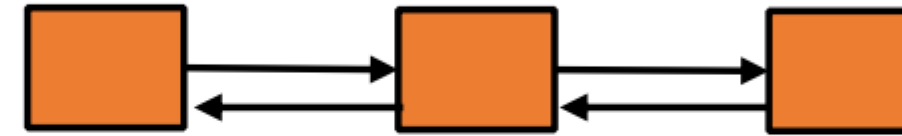
Step 0:



Step 1:



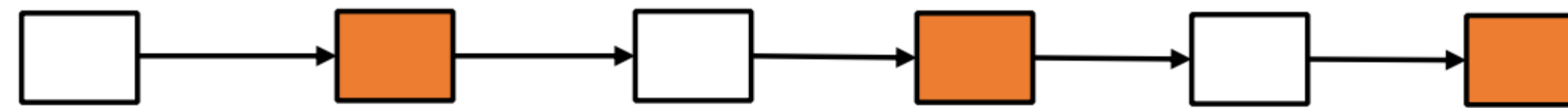
Step 2:



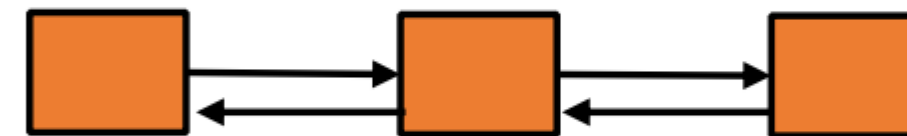
- Option 1: discard nothing
 - Memory +, compute --
- Option 2: discard all and recompute for each layer
 - Memory --, compute++
- We want to strike a balance?

Reduce activation memory

Forward computation



Gradient per segment
with re-computation



For a N layer neural network,
if we checkpoint every K layers

$$\text{Memory cost} = O\left(\frac{N}{K}\right) + O(K)$$

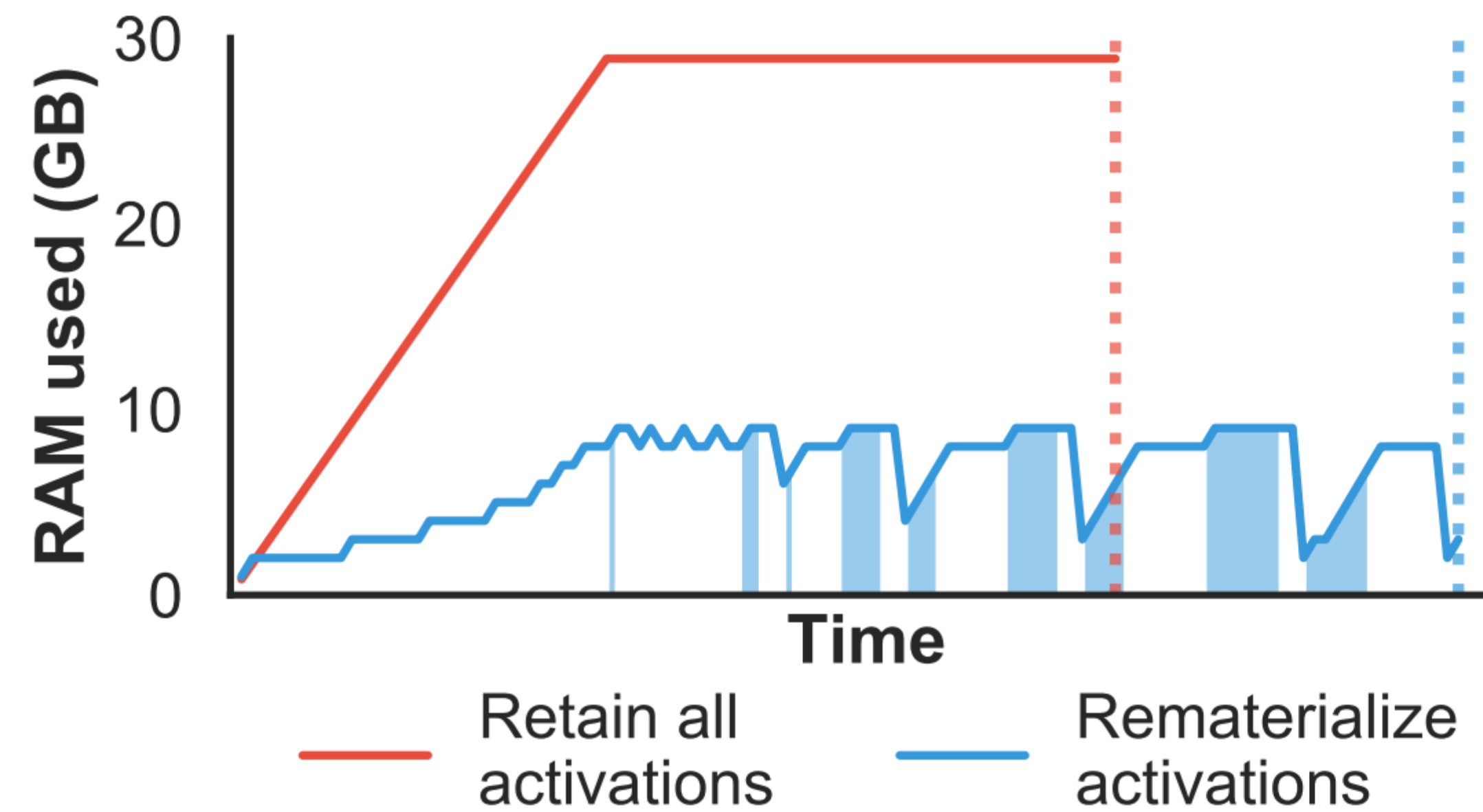
Pick $K = \sqrt{N}$

Checkpoint cost

Re-computation cost

Q: In this case: what is the total recomputation

Memory dynamics



In practice

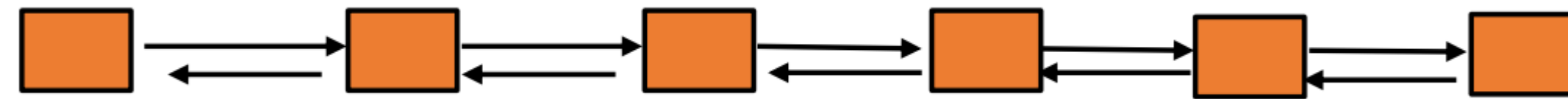
torch.utils.checkpoint

• NOTE

Checkpointing is implemented by rerunning a forward-pass segment for each checkpointed segment during backward propagation. This can cause persistent states like the RNG state to be more advanced than they would without checkpointing. By default, checkpointing includes logic to juggle the RNG state such that checkpointed passes making use of RNG (through dropout for example) have deterministic output as compared to non-checkpointed passes. The logic to stash and restore RNG states can incur a moderate performance hit depending on the runtime of checkpointed operations. If deterministic output compared to non-checkpointed passes is not required, supply `preserve_rng_state=False` to `checkpoint` or `checkpoint_sequential` to omit stashing and restoring the RNG state during each checkpoint.

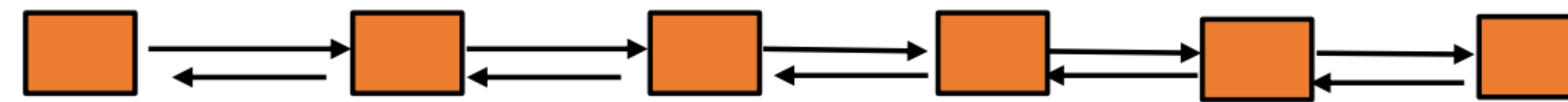
```
"activation_checkpointing": {  
  "partition_activations": false,  
  "cpu_checkpointing": false,  
  "contiguous_memory_optimization": false,  
  "number_checkpoints": null,  
  "synchronize_checkpoint_boundary": false,  
  "profile": false  
}
```


Discussion of Activation Checkpointing



- It is also called: rematerialization, recomputation
- Q: When to and not to enable it?
- Q: Optimal checkpointing policy
 - Which layer to checkpoint at?
 - Could influence memory cost because layer out has different sizes
 - Could influence the recompute cost because the computation between two checkpoints could be different
- Cons: Only applies to activations!

Gradient Accumulation



- 💡 Activation memory is linear to batch size
- Can we still compute using the given batch size but with limited memory

$$\nabla\theta = \nabla_L(\{x_i\}_i^{bs}, \dots)$$
$$\theta = \theta + \nabla\theta$$

vs.

for $t = 1 \rightarrow \#mb$

$$\nabla\theta += \nabla_L(\{x_i\}_i^{mb}, \dots)$$
$$\theta = \theta + \nabla\theta$$

Gradient Accumulation in Practice

```
optimizer = ...

for epoch in range(...):
    for i, sample in enumerate(dataloader):
        inputs, labels = sample

        # Forward Pass
        outputs = model(inputs)
        # Compute Loss and Perform Back-propagation
        loss = loss_fn(outputs, labels)
        loss.backward()
        # Update Optimizer
        optimizer.step()

    optimizer.zero_grad()
```

```
optimizer = ...
NUM_ACCUMULATION_STEPS = ...

for epoch in range(...):
    for idx, sample in enumerate(dataloader):
        inputs, labels = sample

        # Forward Pass
        outputs = model(inputs)
        # Compute Loss and Perform Back-propagation
        loss = loss_fn(outputs, labels)

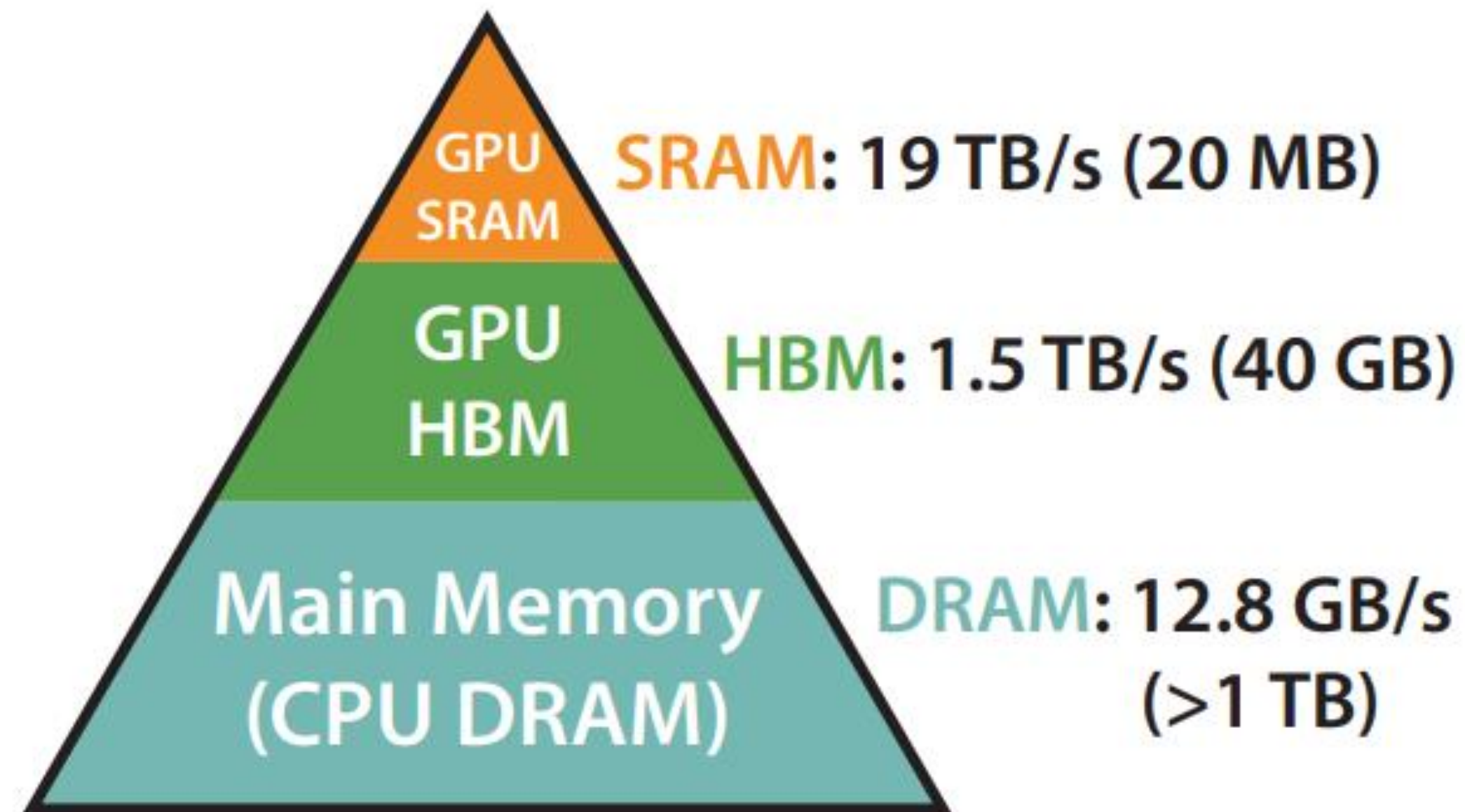
        # Normalize the Gradients
        loss = loss / NUM_ACCUMULATION_STEPS
        loss.backward()

    if ((idx + 1) % NUM_ACCUMULATION_STEPS == 0) or (idx + 1 == len(dataloader)):
        # Update Optimizer
        optimizer.step()

    optimizer.zero_grad()
```

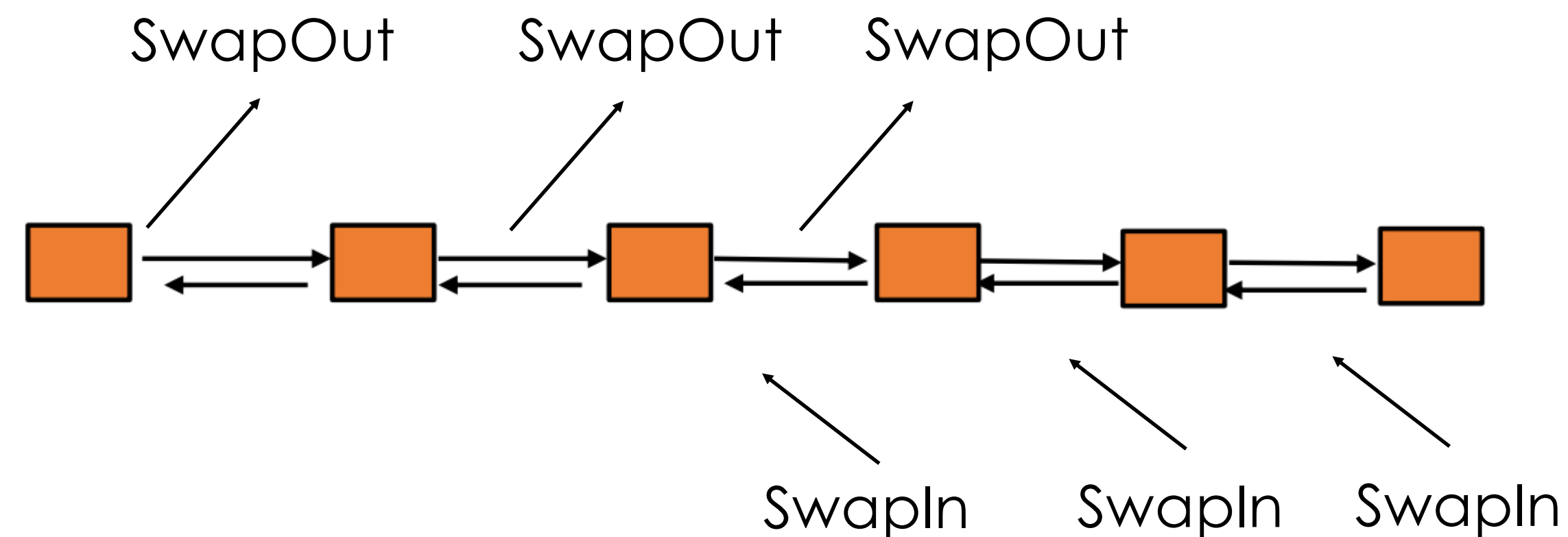
Q: potential cons of gradient accumulation?

Alternative Method: Move to DRAM



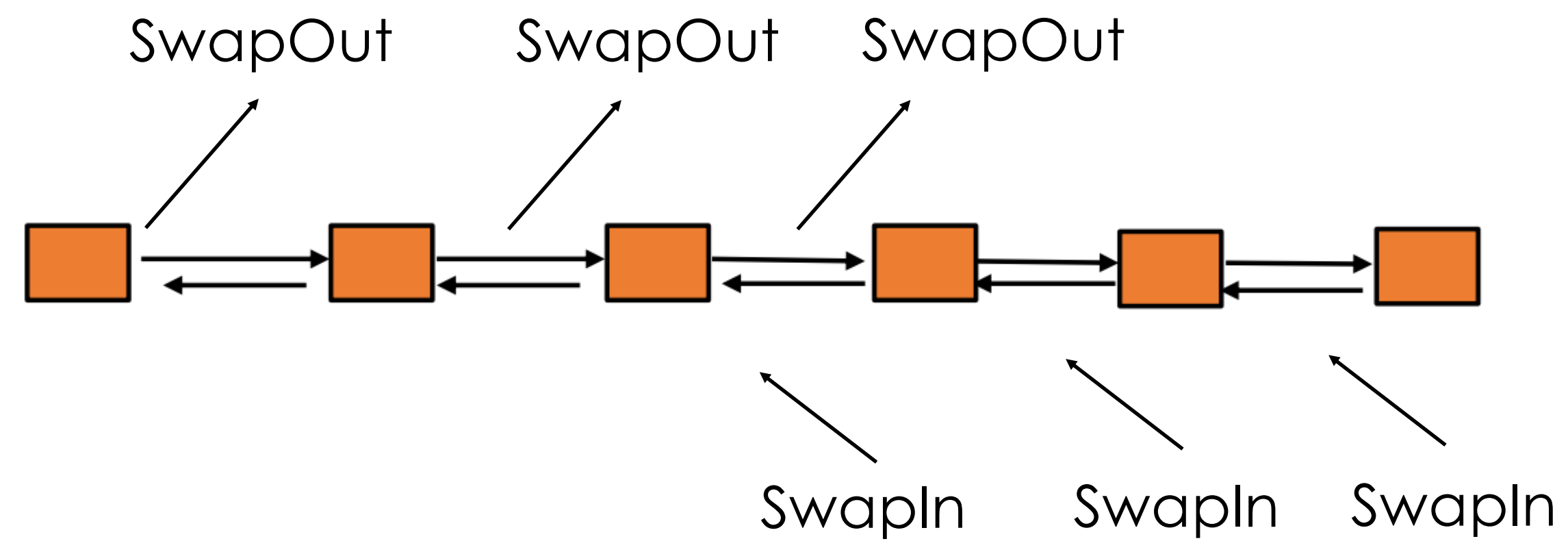
CPU Swap

- SwapIn: swap from CPU DRAM to HBM
- SwapOut: swap from HBM to CPU DRAM
- This applies to both weights and activations!



Discussion

- When will this work and when will this not work?



Summary of Memory Optimizations

- Gradient checkpointing
 - Trading compute for memory
- Gradient accumulation
 - Reduce “effective” batch size
- CPU Swapping
 - Use lower-level memory hierarchy